# СОДЕРЖАНИЕ

Введение ..... 9
1 Анализ литературных исследований и программных решений ..... 11
1.1 Описание и анализ предметной области ..... 11
1.2 Сравнительный обзор аналогов программного средства ..... 13
1.3 Обзор целевых платформ разработки ..... 17
1.4 Обзор архитектурных стилей программных систем ..... 19
1.5 Обоснование выбора технологий разработки ..... 23
1.6 Требования к проектируемому программному средству ..... 30
2 Моделирование предметной области ..... 33
2.1 Описание функциональности программного средства ..... 33
2.2 Разработка функциональных требований к программному средству ..... 34
2.3 Описание коллекций базы данных ..... 34
2.4 Функциональная спецификация требований к программному средству 37
3 Проектирование и разработка программного средства ..... 40
3.1 Проектирование и разработка серверного приложения ..... 40
3.2 Проектирование и разработка клиентского приложения ..... 44
3.3 Развертывание приложения ..... 48
4 Тестирование программного средства ..... 50
4.1 Ручное тестирование программного средства ..... 50
4.2 Тестирование компонентов пользовательского интерфейса ..... 52
5 Методика использования разработанного программного средства ..... 54
6 Технико-экономическое обоснование разработки и реализации на рынке платформы для анализа статистики игрокав компьютерной игре DOTA2 ..... 61
6.1 Характеристика программного средства, разрабатываемого для реализации на рынке ..... 61
6.2 Расчёт инвестиций в разработку программного средства ..... 62
6.3 Расчёт экономического эффекта от реализации программного средства на рынке ..... 65
6.4 Расчёт показателей экономической эффективности разработки и реализации программного средства на рынке ..... 67
6.5 Вывод об экономической целесообразности реализации проектного решения ..... 67
Заключение ..... 69
Список использованных источников ..... 70
Приложение А (обязательное) Антиплагиат ..... 72

Приложение Б (обязательное) Исходный код программы ..... 73

# ОПРЕДЕЛЕНИЯ И СОКРАЩЕНИЯ

В настоящей пояснительной записке применяются следующие определения и сокращения.

Киберспорт – командное или индивидуальное соревнование на основе компьютерных игр.

Геймплей – компонент игры, отвечающий за взаимодействие игры и игрока. Геймплей описывает, как игрок взаимодействует с игровым миром.

API – это набор правил и инструментов, позволяющий программным приложениям взаимодействовать друг с другом.

MOBA – это поджанр компьютерных игр, где две команды игроков противостоят друг другу на структурированной карте, стремясь уничтожить базу противника.

SteamCharts – это платформа, которая отслеживает и визуализирует данные о статистике игроков в Steam.

KDA – композитный показатель эффективности игрока, отражающий его вклад в убийства героев противника (Kills), количество смертей (Deaths) и участие в убийствах (Assists).

GPM – количество золота, зарабатываемого игроком за одну минуту матча.

XPM – количество опыта, зарабатываемого игроком за одну минуту матча.

MMR – это числовой рейтинг, отражающий уровень мастерства игрока в соревновательном режиме.

Герой – это персонаж, которого выбирает игрок для управления в бою.

ИИ – технология, которая позволяет машинам демонстрировать человекоподобные рассуждения и возможности, такие как автономное принятие решений.

ОС – операционная система.

Репликация – это процесс создания и поддержания копий данных или объектов в разных местах.

Шардирование – это метод распределения данных между несколькими отдельными частями, называемыми шардами.

Программное средство (ПС) – компьютерная программа, либо совокупность связанных программ, предназначенная для автоматизации определенной области профессиональной деятельности.

ООП – это парадигма программирования, которая подразумевает разработку программных систем, основанную на создании объектов, обладающих своими данными и функциональностью.

Фреймворк – программное обеспечение, облегчающее разработку и объединение различных компонентов большого программного проекта.

Язык программирования – формальная знаковая система, предназначенная для записи компьютерных программ.

ПО – программное обеспечение.

Крипы – управляемые ИИ существа, которые участвуют в сражениях на линиях и в лесу, и являются источником опыта и золота для героев.

Логи – текстовые файлы, где автоматически записываются события, происходящие в компьютерной системе.

Эндпоинт – адрес, по которому происходит взаимодействие между клиентом и сервером.

Лайнинг – это стадия игры, где герои находятся на линиях и сражаются друг с другом за ресурсы и контроль над территорией.

Стор – это центральный объект, который управляет состоянием приложения и содержит все данные.

# ВВЕДЕНИЕ

Киберспорт пережил колоссальный рост за последние три десятилетия, превратившись в многомиллиардную индустрию с глобальной потребительской базой. В 2021 году доходы отрасли достигли 1,22 миллиарда долларов, и прогнозируется ее дальнейшее значительное расширение. Профессиональные киберспортивные соревнования привлекают многочисленную международную аудиторию и отличаются значительными финансовыми призами. Киберспорт перестал быть нишевым увлечением и прочно утвердился в мейнстриме развлечений, напрямую конкурируя с традиционными видами спорта за внимание зрителей и финансовые потоки. Ожидается, что к 2025 году глобальная аудитория киберспорта превысит 640 миллионов человек. Развитие киберспорта, подкрепленное впечатляющими показателями зрительской аудитории и финансовой прибылью, делает его важной областью для академических исследований и технологических инноваций. Масштаб индустрии указывает на существенный потенциальный рынок для платформы, специализирующейся на анализе данных [1].

Dota 2 выделяется как одна из наиболее сложных и стратегически глубоких дисциплин в киберспорте. Геймплей этой игры требует от участников глубокого понимания командного взаимодействия, тонкостей противостояния героев и высокого уровня стратегического мышления. Для достижения успеха в Dota 2 игрокам необходимо демонстрировать слаженную командную работу, обладать всесторонним знанием карты и принимать взвешенные стратегические решения в режиме реального времени. Сложная механика игры и многообразие способностей героев обуславливают ее исключительную стратегическую глубину. Dota 2 имеет богатую историю на профессиональной киберспортивной сцене и пользуется поддержкой преданного глобального сообщества. Эта многолетняя история и вовлеченность сообщества подтверждают ее значимость как ключевой киберспортивной дисциплины. Внутренняя сложность и стратегическая насыщенность Dota 2 создают значительный спрос на аналитические инструменты и ресурсы, способные помочь игрокам и командам разобраться в тонкостях игры и оптимизировать свою эффективность. Устоявшееся и активное сообщество представляет собой обширную целевую аудиторию для подобной платформы [2].

В ответ на растущее значение анализа данных в киберспортивной сфере, и, в частности, в сложной экосистеме Dota 2, данный дипломный проект направлен на разработку специализированной платформы. Эта платформа

будет посвящена всестороннему анализу статистики игроков в Dota 2. Используя возможности доступных источников данных и применяя передовые современные технологии, проект стремится предоставить пользователям практические знания, которые могут быть непосредственно использованы для улучшения их игрового процесса и способствовать более глубокому пониманию широкого конкурентного ландшафта Dota 2.

Ключевыми требованиями к данной платформе являются удобство использования, интерактивный и интуитивно понятный интерфейс, интеграция с инструментами искусственного интеллекта, а также высокая степень надежности. Автоматизированный анализ с использованием искусственного интеллекта игрока и матчей с его участием позволяет улучшить игровые умения пользователя, делая при этом процесс интерактивным и интересным.

Дипломный проект выполнен самостоятельно, проверен в системе «Атиплагиат». Процент оригинальности соответствует норме, установленной кафедрой информатики. Цитирования обозначены ссылками на публикации, указанные в «Списке использованных источников». Процент оригинальности работы представлен в приложении А.

# 1 АНАЛИЗ ЛИТЕРАТУРНЫХ ИССЛЕДОВАНИЙ И ПРОГРАММНЫХ РЕШЕНИЙ

Результативность программного проекта начинается еще на этапе его подготовки. Важно заранее определить все нюансы и особенности проекта, чтобы минимизировать риски. Первым шагом является четкое формулирование проблемы, которую должна решить система. Это позволяет снизить неопределенности и адекватно планировать проект.

Одним из ключевых факторов успеха является правильная постановка требований. Нечетко сформулированные требования могут привести к увеличению времени и затрат на проект, а также к неудовлетворенности конечных пользователей. Поэтому перед формулированием требований необходимо тщательно проанализировать вопросы выбора платформы и архитектуры системы. На основе этого анализа составляется техническое задание, служащее основой для функциональных требований.

Качество разработки напрямую влияет на успешность проекта. Оно включает в себя стабильность работы приложения, отсутствие ошибок, высокую производительность и защиту данных. Для этого важно придерживаться передовых методик программирования и использовать современные технологии. Например, учет безопасности и производительности является критически важным аспектом, включающим меры по защите данных и оптимизации работы приложения.

Также важен анализ и понимание потребностей целевой аудитории. Инновационная концепция, удобный дизайн и качественный пользовательский опыт играют ключевые роли. Приложение должно быть понятным и удобным в использовании, чтобы удовлетворить требования пользователей и выделиться среди конкурентов.

## 1.1 Описание и анализ предметной области

Предметной областью данного дипломного проекта является анализ игровой статистики в компьютерной игре Dota 2. Проект направлен на создание платформы для анализа статистики игрока в компьютерной игре Dota 2. Эта платформа будет предоставлять игрокам не только агрегированные данные, но и глубокий, контекстуальный анализ их производительности, выявлять сильные и слабые стороны, а также предлагать персонализированные рекомендации для улучшения игрового процесса.

Проект призван обеспечить централизованный и удобный доступ к комплексной статистике, преодолевая ограничения официального API и предлагая более глубокую интерпретацию данных, что позволит игрокам Dota 2 более эффективно анализировать свою игру, принимать обоснованные решения и, в конечном итоге, повышать свой уровень мастерства.

Компьютерные игры прошли долгий путь от простого развлечения для избранных до доминирующей силы в современной культуре. Сегодня индустрия видеоигр представляет собой многомиллиардный рынок, который по своим масштабам успешно конкурирует с такими гигантами индустрии развлечений, как кино и музыка. При этом аудитория геймеров значительно расширилась, охватывая людей всех возрастов и профессий, а не только подростков [3].

Значительным шагом в эволюции многопользовательских игр стало появление возможности сетевой игры через локальную вычислительную сеть, впервые реализованной в игре Doom 2. Это нововведение коренным образом изменило представление об игровом процессе, позволив игрокам соревноваться не только с искусственным интеллектом, но и с реальными соперниками в режиме реального времени. Таким образом, в компьютерных играх зародился элемент спортивной состязательности, который впоследствии привел к формированию киберспорта [4].

Современным эталоном киберспортивной дисциплины по праву считается Dota 2 – MOBA, разработанная компанией Valve Corporation и выпущенная в 2013 году. Этот жанр характеризуется глубокой стратегической составляющей, необходимостью быстро принимать решения и слаженным командным взаимодействием. С момента своего релиза Dota 2 превратилась в настоящий культурный феномен, проникнув в различные формы медиа и став одной из самых влиятельных игр в своем жанре.

С течением времени популярность Dota 2 неуклонно росла. Десятки миллионов игроков ежемесячно заходят в игру, чтобы соревноваться, находить новых друзей и получать удовольствие. Киберспортивная сцена Dota 2 особенно развита, с многочисленными турнирами и лигами, привлекающими лучших игроков со всего мира. Крупнейшим ежегодным событием является The International, который отличается рекордными призовыми фондами и привлекает миллионы зрителей.

Несмотря на конкуренцию со стороны таких проектов, как League of Legends и других MOBA-игр, Dota 2 остается одной из самых популярных и влиятельных игр в своем жанре. По данным SteamCharts на апрель 2025 года, ежемесячная аудитория игры составляет около 393 тысяч активных игроков, а

пиковое количество пользователей, одновременно находящихся в игре, достигало 630 тысяч по данным за тот же период. Эти цифры свидетельствуют о сохраняющейся популярности и значимости Dota 2 в мировом игровом сообществе [5].

## 1.2 Сравнительный обзор аналогов программного средства

Суть сравнительного анализа аналогов разрабатываемого программного средства заключается в выделении их достоинств и недостатков для последующего определения функциональных возможностей, которые необходимо реализовать в разрабатываемом веб-приложении. Сравнительный анализ позволяет учесть опыт существующих решений, выявить их сильные стороны и определить направления для инноваций.

Разрабатываемое решение является платформой для анализа статистики игрока в компьютерной игре Dota 2, и на рынке представлено небольшое количество подобных решений: Stratz, Dotabuff, OpenDota.

Stratz является одной из наиболее продвинутых аналитических платформ для Dota 2, предоставляющей углубленную статистику матчей и игроков. Интерфейс Stratz стремится быть информативным, предлагая пользователям обширные данные. Пример интерфейса Stratz приведен на рисунке 1.2.1.

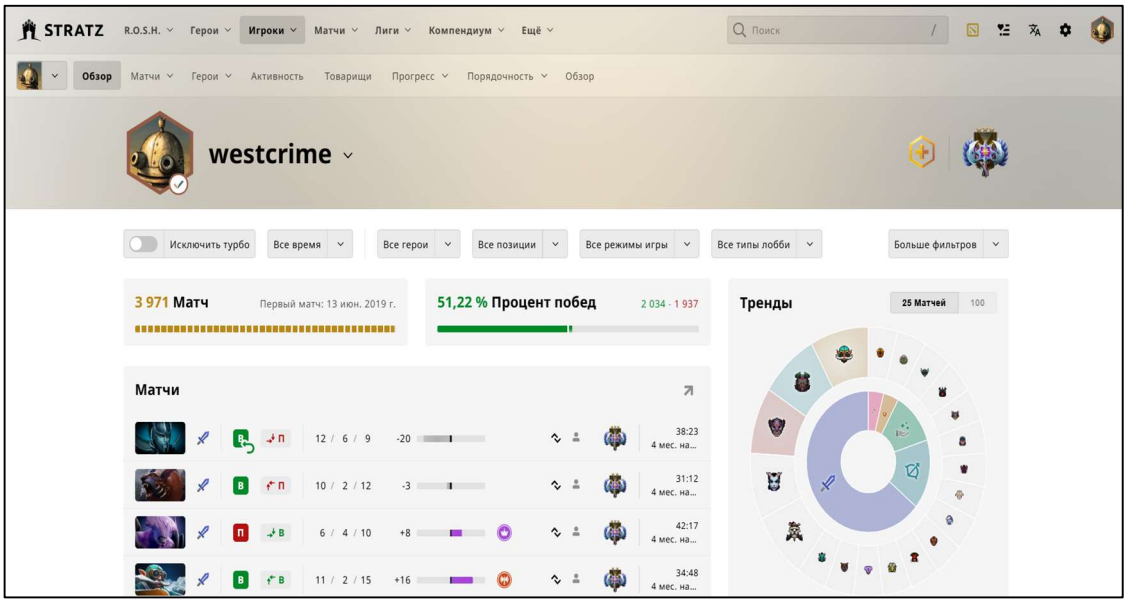



Рисунок 1.2.1 – Интерфейс платформы Stratz

Ключевые аналитические возможности Stratz включают в себя:

– детальная статистика матчей. Метрики (KDA, GPM, XPM, процент побед) для оценки эффективности в игре;

– история игр и прогресс игрока. Отслеживание динамики показателей и определение периодов спадов/улучшений;

– эффективность героев и их сборок. Понимание актуальных героев и оптимальных сборок предметов для героев;

– командные показатели и стратегии. Анализ синергии героев, эффективности совместных действий и общей тактики команды.

Преимущества Stratz:

– высокая детализация аналитики. Глубокий анализ каждого аспекта игры благодаря обширным данным;

– визуализация данных. Удобные графики и таблицы делают сложную информацию понятной;

– наличие API для разработчиков. Открытый интерфейс позволяет создавать сторонние аналитические инструменты.

Недостатки Stratz:

– отсутствие персонализированных рекомендаций на основе ИИ. Нет индивидуальных советов по улучшению игры, основанных на машинном обучении;

– ограниченный анализ слабых мест игрока. Платформа показывает «что» произошло, но не объясняет «почему» и «как» исправить;

– сложный интерфейс для новичков. Множество метрик и опций могут быть трудны для освоения неопытным пользователям.

Dotabuff является одним из старейших и наиболее популярных сервисов статистики для Dota 2, предлагая более базовый, но доступный уровень аналитики. Пример интерфейса Dotabuff приведен на рисунке 1.2.2. Его основной принцип – простота и легкость в освоении.

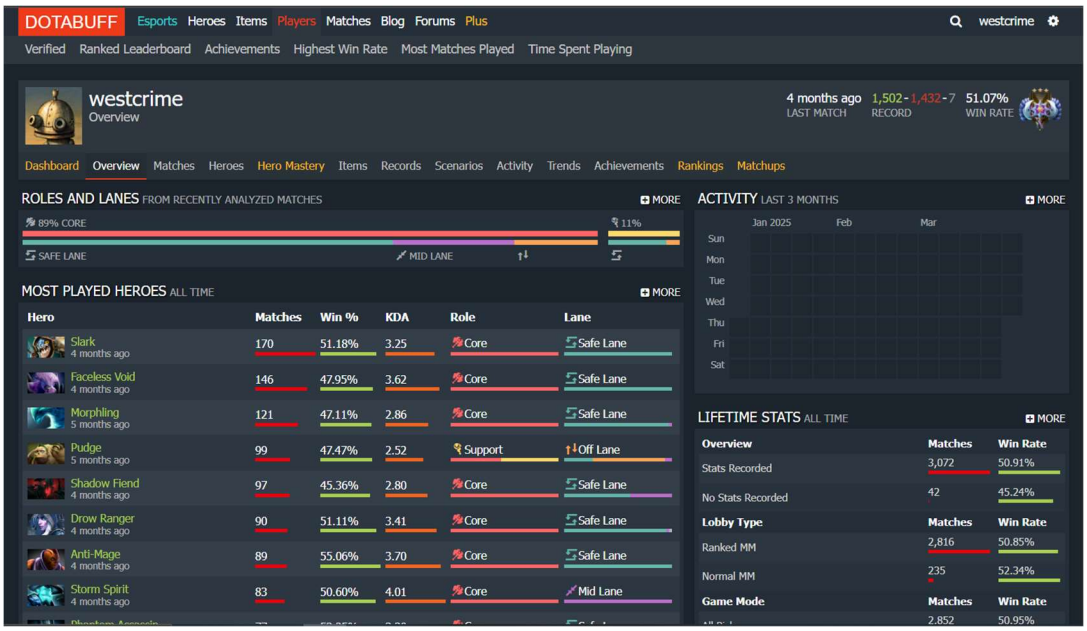



Рисунок 1.2.2 – Интерфейс платформы Dotabuff

Dotabuff предоставляет следующую информацию:

– общая статистика игрока. Суммарные показатели (убийства, смерти, победы) и средние значения метрик;

– рейтинги героев и предметов. Отслеживание их популярности и эффективности в текущем балансе;

– история матчей. Хронологический список игр с основными результатами для быстрого поиска.

Преимущества Dotabuff:

– простота использования. Интуитивный интерфейс для быстрого доступа к базовым метрикам, идеально для новичков;

– доступность базовых метрик. Предоставляет основные показатели без перегрузки информацией.

Недостатки Dotabuff:

– отсутствие углубленного анализа. Недостаточная детализация для глубокого понимания микромоментов и сложных взаимодействий;

– нет рекомендаций по улучшению игры. Отсутствие персонализированных советов или автоматизированных предложений;

– меньшая детализация данных по сравнению с конкурентами. Недостаточен для профессионального или глубокого самоанализа по сравнению с конкурентами.

OpenDota – это открытая платформа для анализа матчей Dota 2, которая выделяется своей ориентацией на сообщество и предоставлением обширного

API. Пример интерфейса OpenDota приведен на рисунке 1.2.3. Открытый характер платформы способствует развитию сторонних инструментов и исследованию данных.

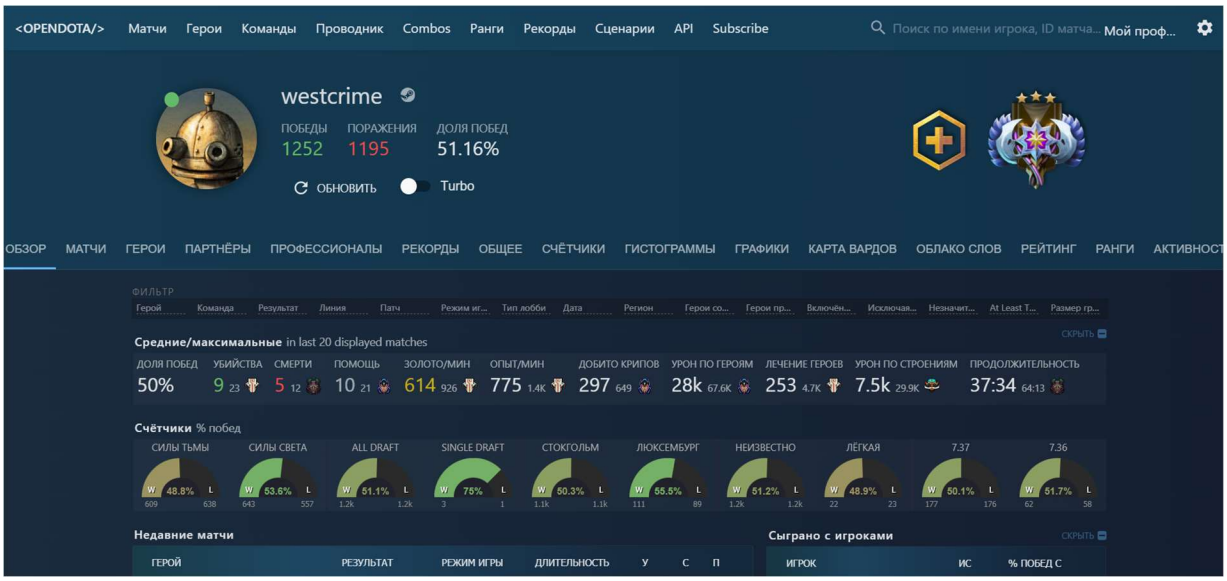



Рисунок 1.2.3 – Интерфейс платформы OpenDota

OpenDota включает:

– разбор матчей по таймлайну. Детальная хронология событий в игре (убийства, предметы, способности) с привязкой ко времени;

– расширенная статистика по героям и игрокам. Глубокие данные по игрокам и героям в разных сценариях;

– инструменты для разработчиков. Мощный открытый API для создания собственных аналитических приложений.

Преимущества OpenDota:

– открытый доступ к данным. Способствует инновациям и новым аналитическим решениям;

– гибкость и кастомизация отчетов. Позволяет создавать индивидуальные запросы и отчеты;

– активное сообщество. Обеспечивает постоянное развитие и улучшение платформы.

Недостатки OpenDota:

– сложный интерфейс. Перегружен для обычных пользователей без технического опыта;

– нет автоматизированных советов для игроков. Фокусируется на данных, но не предлагает рекомендаций для улучшения игры (требуется самостоятельная интерпретация);

– фокус на сырых данных. Обилие необработанной информации может быть менее полезно для рядового пользователя.

Проведенный анализ современных платформ для анализа игровой статистики в Dota 2 выявил ряд характерных тенденций и проблем в данной предметной области. Большинство существующих решений предлагают базовый набор аналитических функций, однако их реализация часто страдает либо избыточной сложностью интерфейса, либо ограниченностью возможностей для глубокого анализа игровых данных.

Облачные платформы, такие как Stratz и Dotabuff, обеспечивают удобный доступ к базовой статистике матчей и минимальный порог входа для новых пользователей. Однако их возможности по персонализированному анализу слабых мест игрока и генерации рекомендаций остаются крайне ограниченными. Открытые системы типа OpenDota предлагают более гибкий доступ к данным через API, но требуют значительных технических знаний для полноценного использования и не предоставляют готовых аналитических решений для обычных игроков.

### 1.3 Обзор целевых платформ разработки

Приступая к разработке любого программного продукта, крайне важно внимательно подойти к выбору технологической платформы. Это решение напрямую влияет не только на ход и стоимость разработки, но и на удобство использования приложения для конечных пользователей, а также на его дальнейшую поддержку и развитие. Неудачный выбор может повлечь за собой серьёзные трудности как на этапе создания, так и в процессе эксплуатации.

Существуют три основные категории, на которых базируется большинство современных приложений: настольные, мобильные и веб-платформы. Каждая из них обладает уникальным набором характеристик, определяющих её пригодность для тех или иных задач:

1 Настольные приложения. Настольные приложения предназначены для установки непосредственно на персональные компьютеры и функционируют под управлением стационарных операционных систем, таких как Windows, macOS или Linux. Их ключевые особенности:

– высокая производительность. Эти приложения способны эффективно обрабатывать значительные объёмы данных и выполнять ресурсоёмкие задачи, используя все доступные мощности компьютера;

– улучшенная безопасность. Работа в локальной, часто изолированной среде операционной системы обеспечивает повышенный уровень защиты данных и конфиденциальности;

– глубокая интеграция с системой. Настольные приложения могут взаимодействовать с аппаратными компонентами и другими программами на более низком уровне, предлагая расширенный функционал.

Однако разработка под настольные платформы требует учёта специфики каждой операционной системы, что может увеличить временные и финансовые затраты на создание и поддержку продукта, поскольку часто необходимо разрабатывать отдельные версии для разных ОС.

2 Мобильные приложения создаются специально для работы на смартфонах и планшетах. Они активно используют уникальные возможности этих устройств, что делает их незаменимыми для многих современных сервисов. Преимуществами разработки под мобильные устройства являются:

– доступ к аппаратным возможностям. Могут использовать встроенные камеры, GPS-модули, акселерометры, биометрические сканеры и другие сенсоры для создания интерактивного и персонализированного пользовательского опыта;

– портативность и постоянная доступность. Приложения всегда под рукой, что обеспечивает мгновенный доступ к информации и функциям в любом месте.

Тем не менее, разработка мобильных приложений сталкивается с ограничениями по производительности и объёму памяти самих устройств, что может влиять на сложность реализуемых функций. Также, как и в случае с настольными, необходимо адаптировать приложение под различные мобильные операционные системы, такие как iOS и Android, что усложняет процесс разработки и тестирования.

3 Веб-приложения функционируют по принципу клиент-серверной архитектуры и доступны пользователям через любой современный веб-браузер. Главными преимуществами данной платформы разработки являются:

– кроссплатформенность. Не требуют установки и могут работать на любой операционной системе и устройстве, где есть веб-браузер, что обеспечивает максимальный охват аудитории;

– простота обновления. Все изменения вносятся централизованно на сервере, избавляя пользователей от необходимости вручную обновлять приложение;

– доступность. Пользователям достаточно иметь подключение к интернету и веб-браузер, чтобы начать работу с приложением.

Основным ограничением веб-приложений является ограниченный доступ к аппаратным возможностям устройства по сравнению с нативными настольными или мобильными приложениями. Это может сужать спектр функций, требующих прямого взаимодействия с оборудованием [6].

При выборе платформы разработки важно учитывать функциональные требования будущего приложения. Каждая платформа имеет свои особенности, которые могут повлиять на процесс разработки и пользовательский опыт. Правильный выбор платформы поможет оптимизировать затраты и сроки разработки, а также улучшить качество конечного продукта.

### 1.4 Обзор архитектурных стилей программных систем

Архитектура играет ключевую роль в разработке программного обеспечения, поскольку именно она определяет структуру и поведение будущей системы. Она служит фундаментом для проектирования, указывая, как отдельные компоненты будут взаимодействовать между собой для достижения заданной функциональности. При этом многообразие доступных архитектурных стилей и паттернов значительно усложняет выбор наиболее подходящего подхода для конкретного проекта или системы.

Архитектурные стили представляют собой высокоуровневые стратегии, которые задают абстрактную структуру для целого семейства систем. Использование определённого архитектурного стиля способствует лучшему разделению ответственностей и повторному использованию проектных решений, часто предлагая готовые подходы для решения типовых задач. Его можно сравнить с общей темой или эстетикой, которая направляет дизайн зданий или жилых домов. В качестве примеров можно привести многослойную (Layered) архитектуру, событийно-ориентированную (Event-Driven) архитектуру и микросервисную (Microservices) архитектуру.

Архитектурные паттерны, в отличие от стилей, более конкретны и применяются для решения специфических проблем внутри определённого модуля или части системы. Они описывают структурированное решение архитектурных задач, детально определяя организацию компонентов и их

взаимодействий для реализации конкретной функциональности. Их можно считать аналогами шаблонов проектирования программного обеспечения, но действующими на более высоком уровне абстракции. Примерами таких паттернов являются Модель-Представление-Контроллер (Model-View-Controller – MVC), издатель-подписчик (Publish-Subscribe) и бессерверные вычисления (Serverless) [7].

Основными архитектурными стилями можно назвать следующие:

1 Монолитная архитектура.

2 Микросервисная архитектура.

3 Клиент-серверная архитектура.

4 Архитектура с использованием серверов обработки событий.

5 Сервис-ориентированная архитектура (SOA).

Монолитная архитектура – это традиционная модель программного обеспечения, которая представляет собой единый модуль, работающий автономно и независимо от других приложений. Это отдельная большая вычислительная сеть с единой базой кода, в которой объединены все бизнес-задачи. Чтобы внести изменения в такое приложение, необходимо обновить весь стек через базу кода, а также создать и развернуть обновленную версию интерфейса, находящегося на стороне службы. Это ограничивает работу с обновлениями и требует много времени.

Монолиты удобно использовать на начальных этапах проектов, чтобы облегчить развертывание и не тратить слишком много умственных усилий при управлении кодом. Это позволяет сразу выпускать все, что есть в монолитном приложении.

К преимуществам монолитной архитектуры можно отнести следующие особенности:

1 Простое развертывание. Использование одного исполняемого файла или каталога упрощает развертывание.

2 Разработка. Приложение легче разрабатывать, когда оно создано с использованием одной базы кода.

3 Производительность. В централизованной базе кода и репозитории один интерфейс API часто может выполнять ту функцию, которую при работе с микросервисами выполняют многочисленные API.

4 Упрощенное тестирование. Монолитное приложение представляет собой единый централизованный модуль, поэтому сквозное тестирование можно проводить быстрее, чем при использовании распределенного приложения.

5 Удобная отладка. Весь код находится в одном месте, благодаря чему становится легче выполнять запросы и находить проблемы.

К недостаткам монолитной архитектуры можно отнести следующие особенности:

1 Снижение скорости разработки. Большое монолитное приложение усложняет и замедляет разработку.

2 Масштабируемость. Невозможно масштабировать отдельные компоненты.

3 Надежность. Ошибка в одном модуле может повлиять на доступность всего приложения.

4 Препятствия для внедрения технологий. Любые изменения в инфраструктуре или языке разработки влияют на приложение целиком, что зачастую приводит к увеличению стоимости и временных затрат.

5 Недостаточная гибкость. Возможности монолитных приложений ограничены используемыми технологиями.

6 Развертывание. При внесении небольшого изменения потребуется повторное развертывание всего монолитного приложения.

Микросервисная архитектура (или просто «микросервисы») представляет собой метод организации архитектуры, основанный на ряде независимо развертываемых служб. У этих служб есть собственная бизнес-логика и база данных с конкретной целью. Обновление, тестирование, развертывание и масштабирование выполняются внутри каждой службы. Микросервисы разбивают крупные задачи, характерные для конкретного бизнеса, на несколько независимых баз кода. Микросервисы не снижают сложность, но они делают любую сложность видимой и более управляемой, разделяя задачи на более мелкие процессы, которые функционируют независимо друг от друга и вносят вклад в общее целое.

Микросервисы дают следующие преимущества:

1 Гибкое масштабирование. Позволяет быстро увеличивать ресурсы для отдельных служб при росте нагрузки, поддерживая высокую производительность.

2 Легкость обслуживания и тестирования. Упрощает эксперименты с функциями, откат версий и быстрое исправление ошибок в отдельных модулях.

3 Независимое развертывание. Позволяет быстро обновлять отдельные функции без влияния на всю систему.

4 Гибкость технологий. При использовании архитектуры микросервисов команды могут выбирать инструменты с учетом своих предпочтений.

5 Высокая надежность. Развертывая изменения для конкретной службы, можно не бояться, что приложение выйдет из строя целиком.

К недостаткам микросервисов можно отнести следующие особенности:

1 Разрастание процесса разработки. Увеличение количества служб и команд может замедлять разработку, если нет должного контроля.

2 Экспоненциальный рост расходов на инфраструктуру. Каждый новый микросервис требует своих затрат на тестирование, развертывание, хостинг и мониторинг.

3 Дополнительные организационные расходы. Требуется усиленная координация и коммуникация между командами.

4 Проблемы при отладке. Множество разрозненных журналов и распределенный характер бизнес-процессов усложняют поиск и исправление ошибок.

5 Отсутствие стандартизации. Без общей платформы может возникнуть ситуация, в которой расширяется список языков, стандартов ведения журналов и средств мониторинга.

6 Отсутствие ясности в вопросах владения. С ростом числа сервисов становится сложнее определить ответственность за их поддержку [8].

Клиент-серверная архитектура – это модель организации вычислительных систем, в которой задачи распределены между клиентами и серверами. В такой архитектуре клиент, обычно являющийся пользователем или программой, запрашивает услуги или ресурсы у сервера, который отвечает на запросы, предоставляя необходимые данные или функциональность.

Основные особенности клиент-серверной архитектуры:

1 Разделение функций. Клиент запрашивает, сервер обрабатывает. Это распределяет нагрузку, улучшает масштабируемость и эффективность обработки запросов.

2 Сервер как «черный ящик». Клиент взаимодействует с сервером, не зная его внутренней работы, что упрощает использование.

3 Надежность. Распределение нагрузки между серверами повышает отказоустойчивость: при сбое одного сервера клиенты переключаются на другой.

4 Масштабируемость. Легко добавлять клиентов и серверы, обеспечивая горизонтальное и вертикальное масштабирование для растущих нагрузок.

5 Централизованное управление. Сервер контролирует ресурсы, данные и услуги, упрощая управление системой и обеспечивая единые политики безопасности.

6 Взаимодействие с разными платформами. Использует открытые стандарты для обмена данными, что позволяет взаимодействовать с различными технологиями и платформами [9].

Архитектура, управляемая событием (EDA), – это современный архитектурный шаблон, созданный из небольших несвязанных сервисов, которые публикуют, потребляют или маршрутизируют события.

Событие представляет собой изменение состояния или его обновление. Например, товар помещен в корзину, файл загружен в систему хранения данных или заказ, который становится готовым к отправке. События могут либо сохранять состояние (например, название товара, цена или его количество в заказе) или просто содержать идентификаторы (например, «заказ № 8942 отправлен») для поиска соответствующей информации.

В отличие от традиционных моделей, управляемых запросами, EDA обеспечивает слабую связанность между сервисами источника и приемника. Благодаря этому становится проще масштабировать обновлять и развертывать отдельные компоненты системы независимо друг от друга.

Архитектура, управляемая событиями (EDA), способствует слабой взаимозависимости между компонентами системы, что приводит к большей гибкости. Микросервисы могут масштабироваться независимо, выходить из строя без влияния на другие сервисы и снижать сложность рабочих процессов. События можно гибко маршрутизировать, буферизировать и регистрировать для целей аудита. Потоки событий на основе технологии push могут работать в режиме реального времени, сокращая расходы, связанные с созданием и эксплуатацией кода, который постоянно опрашивает системы на предмет изменений [10].

Сервис-ориентированная архитектура – это подход к разработке программного обеспечения, при котором приложение строится из набора слабосвязанных, автономных компонентов, называемых сервисами. Каждый такой сервис представляет собой логически обособленную, повторяющуюся бизнес-функцию или операцию с четко определенным интерфейсом. Главная идея SOA заключается в создании компонентов, которые могут быть легко переиспользованы и взаимодействовать друг с другом, независимо от их внутренней реализации, платформы или языка программирования.

### 1.5 Обоснование выбора технологий разработки

Для разработки проекта выбрана классическая клиент-серверная архитектура с четким разделением компонентов:

– frontend: клиентская часть;

– backend: серверная часть, обеспечивающая бизнес-логику приложения;

– база данных: хранение данных проанализированных матчей;

При выборе технологий для разработки учитывались такие факторы как: скорость разработки, масштабируемость, производительность, безопасность. В данном документе рассматривается обоснование выбора технологического стека для проекта.

### **1.5.1** Язык программирования JavaScript

JavaScript – это интерпретируемый язык программирования, который используют для написания frontend- и backend-частей сайтов, а также мобильных приложений. Часто в текстах и обучающих материалах название языка сокращают до JS. Это язык программирования высокого уровня, то есть код на нем понятный и хорошо читается.

JS поддерживают все популярные браузеры. Во frontend-части сайтов язык используют для создания интерактива (анимаций, всплывающих форм, автозаполнения), так как он связан с HTML и CSS и может ими манипулировать. В backend-части с языком JavaScript работают на платформе Node.js. С ее помощью, например, разрабатывают серверные веб-приложения и подключают библиотеки. В поисковике Google на JavaScript работает строка автозаполнения, а Netflix, Uber, eBay используют его в своем backend. Уже 6 лет JS – самый популярный язык среди разработчиков по версии GitHub [11].

### **1.5.2** React

React представляет собой JavaScript-библиотеку с открытым исходным кодом, предназначенную для разработки интерактивных пользовательских интерфейсов. Занимая важное место в современной веб-разработке, React используется для создания как одностраничных приложений, так и сложных веб-платформ, обеспечивая высокую производительность и удобство как для разработчиков, так и для конечных пользователей.

React определяется как декларативная, компонентно-ориентированная JavaScript-библиотека, нацеленная на упрощение процесса создания динамичных и отзывчивых пользовательских интерфейсов для веб- и мобильных приложений. В отличие от некоторых фреймворков, которые могут диктовать определенные способы решения задач, React концентрируется исключительно на уровне представления (view layer), что предоставляет разработчикам свободу в выборе других инструментов, таких

как библиотеки для маршрутизации и управления состоянием. Такой подход делает React более гибким и адаптируемым к различным требованиям проектов.

Ключевые преимущества React включают его декларативный подход, компонентную архитектуру, возможность повторного использования кода и высокую производительность, достигаемую благодаря использованию виртуального DOM. В более подробной форме архитектура React представлена на плакате «Архитектура React приложения» (см. перечень графического материала). Декларативное программирование позволяет разработчикам описывать желаемый результат, не вдаваясь в детали реализации, что делает код более понятным и легким в отладке. Компонентная архитектура способствует созданию модульных и независимых частей пользовательского интерфейса, которые могут быть многократно использованы в различных частях приложения, что значительно повышает эффективность разработки и упрощает поддержку кода.

### **1.5.2** Redux

Redux – это контейнер состояния для JavaScript-приложений, разработанный для создания согласованных, легко отлаживаемых приложений, которые работают одинаково в разных средах (клиент, сервер, нативный). Он помогает управлять состоянием приложения, делая его предсказуемым и централизованным. Redux особенно популярен в сочетании с React, но может использоваться с любой другой библиотекой или фреймворком для построения пользовательских интерфейсов.

Схема архитектуры хранилища Redux представлена на плакате «Архитектура хранилища Redux» (см. перечень графического материала).

Redux основан на трёх ключевых принципах, которые обеспечивают его предсказуемость и простоту отладки:

1 Единый источник истины (Single Source of Truth). Весь стейт (состояние) вашего приложения хранится в одном JavaScript-объекте внутри одного стора (хранилища). Это значительно упрощает отладку, так как вам не нужно искать состояние в разных частях приложения. Вы всегда знаете, где искать данные.

2 Состояние доступно только для чтения (State is Read-Only). Единственный способ изменить состояние – это отправить экшен (action) – простой JavaScript-объект, описывающий произошедшее событие. Вы не можете напрямую изменять стейт. Это гарантирует, что изменения состояния происходят предсказуемо и явно.

3 Изменения вносятся чистыми функциями (Changes are Made with Pure Functions). Чтобы указать, как стейт изменяется в ответ на экшены, вы пишете редьюсеры (reducers). Редьюсеры — это чистые функции, которые принимают текущий стейт и экшен, а затем возвращают новый стейт. Они не должны иметь побочных эффектов или изменять исходный стейт напрямую.

### **1.5.3** C#

C# – это современный, объектно-ориентированный, типобезопасный язык программирования, разработанный компанией Microsoft в рамках инициативы .NET. Он был создан для объединения лучших характеристик C++ (мощность, производительность) и Java (простота, безопасность, управление памятью), а также для добавления новых возможностей, специфичных для платформы .NET. C# является одним из наиболее популярных языков для разработки широкого спектра приложений – от настольных и веб-приложений до мобильных, облачных и игровых решений.

Основные характеристики C#:

1 Объектно-ориентированная парадигма (ООП). C# полностью поддерживает принципы ООП: инкапсуляцию, наследование, полиморфизм и абстракцию. Это позволяет создавать модульный, повторно используемый и легко расширяемый код.

2 Типобезопасность. C# является строго типизированным языком, что означает, что все переменные должны иметь явно определенный тип. Это помогает выявлять ошибки на этапе компиляции, а не во время выполнения, что повышает надежность и стабильность приложений.

3 Управляемый код. Программы на C# компилируются в промежуточный язык (IL – Intermediate Language) и выполняются в среде выполнения .NET (Common Language Runtime – CLR). CLR предоставляет такие сервисы, как автоматическое управление памятью (сборка мусора), обработка исключений и проверка безопасности типов, что значительно упрощает разработку и снижает риск ошибок.

4 Кроссплатформенность. Благодаря .NET Core (теперь просто .NET), C# и приложения, написанные на нем, могут работать на различных операционных системах, включая Windows, Linux и macOS, что значительно расширя область его применения.

5 Богатый набор библиотек. C# имеет доступ к обширной библиотеке классов .NET (Base Class Library – BCL), которая предоставляет функциональность для работы с файловой системой, сетью, базами данных, графикой, XML и многим другим.

6 Современные языковые особенности. C# постоянно развивается, добавляя новые возможности, такие как LINQ (Language Integrated Query) для работы с данными, асинхронные операции (async/await) для повышения отзывчивости, делегаты, события, лямбда-выражения и многое другое.

7 Интеграция с экосистемой Microsoft. C# тесно интегрирован с технологиями Microsoft, такими как ASP.NET (веб-разработка), Xamarin/MAUI (мобильная разработка), WPF/WinForms (настольные приложения), Azure (облачные сервисы) и Unity (разработка игр) [12].

### **1.5.4** ASP.NET Core

ASP.NET Core – это современный, высокопроизводительный, кроссплатформенный и открытый веб-фреймворк от Microsoft, предназначенный для создания облачных, масштабируемых, подключенных к интернету приложений. Он является переосмыслением и эволюцией традиционного ASP.NET, разработанным с нуля для удовлетворения требований современных веб-технологий и архитектур. ASP.NET Core является частью более широкой экосистемы .NET (после унификации .NET Core и .NET Framework, сейчас это просто .NET) и позволяет разработчикам использовать мощь языка C# для создания серверной логики.

ASP.NET Core выделяется рядом характеристик, которые делают его предпочтительным выбором для современной веб-разработки:

1 Кроссплатформенность и открытый исходный код. Приложения могут быть разработаны и запущены на Windows, macOS и Linux, а также в контейнерах Docker. Фреймворк полностью открыт (доступен на GitHub), что способствует активному участию сообщества, прозрачности разработки и быстрому внедрению инноваций.

2 Высокая производительность и масштабируемость. Фреймворк Разработан с акцентом на производительность и показывает отличные результаты в бенчмарках. Это критично для высоконагруженных веб-сервисов и API.

3 Асинхронные операции. Широкое использование асинхронных паттернов (async/await) позволяет эффективно обрабатывать множество запросов без блокировки потоков, что улучшает масштабируемость.

4 Универсальность. Позволяет создавать различные типы веб-приложений.

5 Встроенная инъекция зависимостей (DI). Фреймворк имеет встроенный контейнер для управления зависимостями, что поощряет написание модульного кода.

6 Модульность. ASP.NET Core построен на модульной основе. пользователь добавляет только те компоненты и библиотеки, которые нужны приложению, что уменьшает его размер и повышает производительность.

### **1.5.5** MongoDB

MongoDB – это документоориентированная система управления базами данных, которая не требует описания схемы таблиц. Считается одним из классических примеров NoSQL-систем, использует JSON-подобные документы и схему базы данных. MongoDB имеет открытый исходный код, она бесплатная и доступна любому разработчику. СУБД подходит для операционных систем семейства Linux, Windows и macOS.

В реляционной базе данных информация представлена как набор связанных между собой таблиц. Доступ к данным осуществляется с помощью запросов на языке SQL. Реляционная модель – традиционная, ее используют чаще всего. Но для многих задач она неоптимальна, например, если связи между разными данными не жесткие.

В MongoDB реализована система, при которой данные хранятся в «табличных» документах форматов, близких к JSON. Информация записывается в виде пар из ключей и значений – как в таблице, в которой есть идентификаторы и соответствующие им данные. Благодаря такому формату в MongoDB можно разместить очень разнообразную и сложно структурированную информацию: документ – более гибкая структура [13].

Преимущества использования MongoDB:

1 Гибкость схемы. Возможность быстро адаптироваться к изменяющимся требованиям к данным без трудоемких миграций схемы. Идеально подходит для быстро развивающихся проектов (Agile-разработка) и приложений, где структура данных может меняться со временем.

2 Масштабируемость. Встроенные механизмы репликации и шардирования позволяют легко масштабировать базу данных горизонтально для обработки больших объемов данных и высокой нагрузки.

3 Производительность. Высокая скорость чтения и записи данных, особенно для операций, которые не требуют сложных JOIN-операций, благодаря документоориентированной модели.

4 Простота разработки: Использование JSON-подобных документов упрощает взаимодействие с данными для разработчиков, особенно тех, кто уже знаком с JavaScript.

5 Богатый набор функций. Поддержка геопространственных запросов, полнотекстового поиска, агрегации данных и других мощных функций «из коробки».

6 Активное сообщество и экосистема. Большое и активное сообщество, обширная документация, множество драйверов для различных языков программирования и инструментов.

### **1.5.5** Docker

Docker – это платформа, которая позволяет упаковать в контейнер приложение со всем окружением и зависимостями, а затем доставить и запустить его в целевой системе.

Приложение, упакованное в контейнер, изолируется от операционной системы и других приложений. Поэтому разработчики могут не задумываться, в каком окружении будет работать их приложение, а инженеры по эксплуатации – единообразно запускать приложения и меньше заботиться о системных зависимостях.

Docker разработали в 2008 году. Изначально это был внутренний проект компании dotCloud, которую впоследствии переименовали в Docker Inc. В 2013 году dotCloud открыла исходный код Docker для сообщества.

Виртуализация в Docker реализуется на уровне ОС. Виртуальная среда запускается прямо из ядра основной операционной системы и использует её ресурсы.

Контейнеры и виртуальные машины – это разные способы виртуализации. Только виртуалка реализует её на уровне железа, а Docker – на уровне операционной системы.

Виртуальная машина функционирует как отдельный компьютер с собственным оборудованием и операционной системой. Распространённая практика – купить большой сервер и установить на него гипервизор, базу для виртуалок. Сервер «нарезается» на много виртуальных компьютеров, что избавляет от необходимости покупать их отдельно.

Виртуальные компьютеры вполне полноценны. На них можно установить операционную систему любого семейства и работать в ней, например, через графический интерфейс в многопользовательском режиме, устанавливая и запуская множество приложений и сервисов.

Если цель виртуалки – полностью воспроизвести устройство компьютера, то основная цель Docker – создать среду для одного приложения. Виртуальная среда контейнера запускается внутри операционной системы. Ей не нужно виртуализировать оборудование – она использует его через ОС.

Поэтому контейнеры Docker потребляют меньше ресурсов, быстрее развёртываются, проще масштабируются и меньше весят [14].

## 1.6 Требования к проектируемому программному средству

После анализа предметной области и сравнительного обзора аналогов программного средства необходимо сформулировать требования к проектируемому программному средству.

### **1.6.1** Назначение проекта

Назначением проекта является создание программного средства для автоматизации процесса анализа и улучшения игрового процесса в Dota 2. Основной целью приложения является предоставление игрокам возможности анализировать свои матчи, отслеживать статистику, получать персонализированные рекомендации по улучшению игры.

### **1.6.2** Основные функции

Программное средство должно реализовывать следующие основные функции:

– аутентификация пользователей через Steam;

– отображение списка последних матчей с краткой статистикой и возможной фильтрацией;

– отображение общей информации игрока (имя пользователя, уникальный идентификатор и т.д.);

– отображение общей статистики за все время (процент побед, ранг и т.д.);

– отображение рекордов и средних показателей пользователя (количества убийств, смертей, заработанного опыта);

– отображение уровня игры за последние 10 матчей на различных героях;

– анализ матча, включающий в себя анализ разных аспектов игры и генерация, и отображение советов для улучшения навыков пользователя.

### **1.6.3** Требования к входным данным

Входные данные для программного средства должны быть представлены в виде:

– данных пользователя из Steam (Steam ID, имя пользователя, аватар);

– настроек для фильтрации матчей.

В бизнес-логике ПС должны быть реализованы проверки входных данных на корректность и актуальность. При обнаружении ошибок система должна предоставлять понятные уведомления с описанием проблемы.

**1.6.4** Требования к выходным данным

Выходные данные должны быть представлены в виде:

– общей информации профиля игрока (имя, уникальный идентификатор, аватар и т.д.);

– статистики игрока за все количество матчей (процент побед, ранг и т.д.);

– общих рекордов пользователя и среднего значения (общее количество убийств, помощи, заработанного золота и т.д.);

– анализа уровня игры на различных героях за последние 10 матчей;

– просмотра отфильтрованных последних матчей;

– анализа конкретного матча, включающего анализ разных аспектов с визуализацией и сгенерированным ИИ-ассистентом советом.

**1.6.5** Выбор инструментов разработки

На основании установленных основных функций, анализа доступных технологий и архитектурных подходов, было принято решение о проектировании веб-приложения с клиент-серверной архитектурой.

Для клиентской части:

– Vite как сборщик приложения;

– TypeScript для надежной типизации;

– React для построения пользовательского интерфейса;

– React Router для маршрутизации;

– Redux Toolkit для управления состоянием;

– Axios для работы с API;

– Material UI для компонентов интерфейса;

– Tailwind CSS для стилизации.

Для серверной части:

– .NET как платформа;

– ASP.NET Core для серверной логики;

– MongoDB для основной базы данных;

– OpenRouter API для интеграции с ИИ;

– Steam Web API для интеграции;

– OpenDota API для расширенной статистики;

– Stratz API для расширенной статистики.

Для развёртывания и управления приложением:

– Linux-сервер (Hetzner CX32 или CX42) с 4-8 vCPU, 8 ГБ RAM и 80 ГБ NVMe SSD;

– Docker и Docker Compose для контейнеризации приложения и упрощения развертывания;

– система управления версиями Git;

– среда разработки Visual Studio Code;

– автоматизированное тестирование с использованием Jest для клиентской части;

– CI/CD для автоматизации процесса развертывания.

Выбор описанных технологий и инструментов обеспечит стабильную работу приложения, его масштабируемость и удобство использования. Благодаря применению MongoDb, работа с данными будет удобной и надёжной, использование React.js позволит создать современный и отзывчивый пользовательский интерфейс, а применение ASP.NET Core обеспичит быструю обработку данных на сервере.

# 2 МОДЕЛИРОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ

В данной части дипломной работы будет произведено моделирование предметной области, в рамках которой создается данная дипломная работа.

## 2.1 Описание функциональности программного средства

Для описания функциональности программного средства была выбрана UML диаграмма вариантов использования, которая отражает отношения между актерами и прецедентами и позволяет описать систему на концептуальном уровне. Основные элементы диаграммы – актер и прецедент.

Возможности пользователя представлены на плакате «Use-case диаграмма пользователя» (см. перечень графического материала) в виде диаграммы вариантов использования, разработанной с использованием нотации UML 2.1.

В соответствии с диаграммой можно выделить два основных составляющих элемента: актер и прецедент.

Актер – роль внешнего объекта программного средства, прямо взаимодействующая с ее частями. Участником может быть человек, другая система, подсистема или класс.

Прецедент – описание множества последовательных событий, выполняемых программным средством, а также отображение взаимодействия между участниками и системой.

Пользователь в системе анализа статистики Dota 2 получает доступ к комплексному инструментарию для работы с игровыми данными. После авторизации через Steam-аккаунт и привязки профиля Dota 2, система предоставляет возможность глубокого анализа игровой деятельности.

Основной функционал включает просмотр детализированной статистики аккаунта с отображением уровня, текущего MMR и общего количества сыгранных матчей, а также графиков динамики изменения рейтинга за выбранные периоды времени. Пользователи могут изучать персонализированную статистику по героям, включая показатели винрейта, KDA и эффективности для каждого персонажа, с выделением наиболее успешных героев для текущего уровня игры.

Система предлагает анализ ключевых игровых показателей (GPM, XPM, нанесенный урон, лечение), позволяющий сравнивать текущие результаты с предыдущими периодами. На основе собранных данных формируются

персонализированные рекомендации по улучшению навыков, выделяющие слабые места и предлагающие конкретные способы их устранения.

Дополнительно реализована функция сравнения личных показателей со средними значениями для текущего рейтинга, что помогает пользователям объективно оценивать свой уровень игры и определять приоритетные направления для совершенствования. Все данные могут быть обновлены по запросу для получения актуальной информации с серверов Steam.

## 2.2 Разработка функциональных требований к программному средству

Рассмотрим функциональные требования для данной платформы:

– авторизация в системе через аккаунт Steam;

– просмотр основной информации профиля (никнейм, уникальный идентификатор и т. д.);

– просмотр рекордов пользователя (общее количество убийств, помощи и т. д.);

– анализ уровня игры пользователя на различных героях;

– просмотр основной статистики (общий процент побед, ранг и т. д.);

– просмотр истории матчей с возможной фильтрацией;

– подробный анализ матча на разных этапах, сравнение показателей с показателями других игроков на этом рейтинге, получение советов, сгенерированные на основе ИИ;

## 2.3 Описание коллекций базы данных

MongoDB – это документоориентированная нереляционная СУБД, которая распространяется по лицензии SSPL и имеет открытый исходный код. В отличие от традиционных реляционных баз данных, где данные организованы в таблицы со строгим набором столбцов, MongoDB хранит данные в виде документов, которые, в свою очередь, объединены в коллекции. Такой подход обеспечивает высокую гибкость схемы (schema-less), позволяя адаптироваться к изменяющимся требованиям к данным, что крайне важно для динамичной предметной области компьютерных игр и их патчей. MongoDB гарантирует высокую производительность и масштабируемость, делая её отличным выбором для разрабатываемого ПС.

Одной из ключевых особенностей MongoDB, которая делает ее подходящей для проекта такого вида, является её способность эффективно

работать с вложенными структурами данных и массивами. Это позволяет хранить всю информацию о матче и игроке в одном документе или связанных документах, минимизируя необходимость в сложных JOIN-операциях, характерных для реляционных баз данных [15].

– _id: ObjectId – уникальный идентификатор документа в MongoDB. Автоматически генерируется;

– MatchId: Number (long) – уникальный идентификатор матча;

– DotaId: Number (long) – внутренний идентификатор пользователя в системе, связанный со Steam Account ID.

– GeneralInfo: Object – общая информация о матче (вложенный документ):

– DidRadiantWin: Boolean – показывает, выиграла ли команда Radiant;

– Rank: Number (int) – ранг игроков в матче;

– DurationSeconds: Number (int) – длительность матча в секундах;

– RadiantNetworthLeads: Array of Numbers (int) – преимущество сил света по общей ценности в разные моменты времени;

– RadiantExperienceLeads: Array of Numbers (int) – преимущество сил света по опыту в разные моменты времени;

– RadiantKills: Array of Numbers (int) – количество убийств команды сил света по времени;

– DireKills: Array of Numbers (int) – количество убийств команды сил тьмы по времени;

– Players: Array of Objects – список статистики для каждого игрока в матче (вложенный массив документов):

– Position: String – роль игрока;

– SteamAccountId: String – Steam ID игрока;

– HeroId: Number (int) – ID героя, которым играл игрок;

– IsRadiant: Boolean – принадлежность к команде сил света;

– Networth: Number (int) – ценность игрока;

– Kills, Deaths, Assists: Number (int) – показатели KDA;

– GoldPerMinute, ExperiencePerMinute: Number (int) – GPM и XPM;

– NumDenies, NumLastHits: Number (int) – добитые и не отданные крипы;

– Imp: Number (int) – показатель производительности (Impact);

– Item0Id - Item5Id: Number (int) / null – ID основных предметов игрока (может быть null);

– Backpack0Id - Backpack2Id: Number (int) / null – ID предметов в рюкзаке (может быть null);

– UserStats: Object – детальная статистика анализируемого пользователя в этом матче (вложенный документ):

– HeroId: Number (int) – ID героя, которым играл пользователь;

– IsRadiant: Boolean – был ли пользователь в команде Radiant;

– Networth, Kills, Deaths, Assists, GoldPerMinute, ExperiencePerMinute, NumDenies, NumLastHits, Imp: Number (int) – аналогичные показатели KDA, GPM, XPM и т.д. для анализируемого пользователя;

– ImpactPerMinute: Array of Numbers (int) – динамика влияния пользователя по минутам;

– ItemPurchases: Array of Objects – история покупок предметов пользователем (вложенный массив документов):

– ItemId: Number (int) – ID купленного предмета;

– Time: Number (int) – время покупки предмета в секундах;

– AvgHeroStats: Object – средняя статистика по герою, которым играл пользователь (для сравнения) (вложенный документ AvgHeroStatsEntity):

– TopCore, TopSupport: Number (double) – средние показатели для игроков разных ролей;

– Kills, Deaths, Assists, Networth, Xp, Cs, HeroDamage, GoldFed, XpFed: Number (double) – средние значения соответствующих метрик для этого героя;

– Laning: Object – детализированный анализ фазы лайнинга пользователя (вложенный документ LaningEntity):

– LaneAlliesHeroIds: Array of Numbers (int) – ID союзных героев на линии;

– LaneEnemiesHeroIds: Array of Numbers (int) – ID вражеских героев на линии;

– Position: String – роль игрока;

– Rank: String – ранг игрока в этом матче;

– HeroId: Number (int) – ID героя пользователя;

– DotaId: Number (long) – ID пользователя;

– LaningCs, AvgLaningCs: Number (int/double) – количество добитых крипов на линии и их среднее значение;

– LaningKills, AvgLaningKills: Number (int/double) – убийства на линии и их среднее значение;

– LaningDeaths, AvgLaningDeaths: Number (int/double) – смерти на линии и их среднее значение;

– LaningNetworth, AvgLaningNetworth: Number (int/double) – чистая ценность на линии и её среднее значение;

– Picks: Object – Информация о пиках героев и процентах побед (вложенный документ PicksEntity):

– HeroId: Number (int) – ID героя пользователя;

– HeroWinrate: Number (double) – общий винрейт героя пользователя;

– HeroWrWithAlliedHeroes: Array of Objects – процент побед героя в комбинации с союзниками (вложенный массив HeroWrEntity):

– Winrate: Number (double) – процент побед;

– HeroId: Number (int) – ID союзного героя;

– HeroWrWithEnemyHeroes: Array of Objects – процент побед героя против вражеских героев (вложенный массив HeroWrEntity):

– Winrate: Number (double) – процент побед;

– HeroId: Number (int) – ID вражеского героя;

– WinratesAnalysis: String – текстовый анализ винрейтов (например, рекомендация по героям);

– LaningAnalysis: String – текстовый анализ фазы лайнинга;

– ItemsAnalysis: String – текстовый анализ сборок предметов;

## 2.4 Функциональная спецификация требований к программному средству

С учетом требований, определенных в подразделе 1.6 области можно сформулировать следующие функциональные требования к системе:

1 Авторизация пользователей через Steam должна быть реализована с учетом следующих требований:

– авторизация инициируется по нажатию кнопки;

– процесс авторизации происходит на стороне Steam.

2 Отображение общей статистики за все время:

– отображение текущего ранга игрока;

– отображение общего количества игр;

– отображение процента побед в виде линейной диаграммы;

– отображение даты первого матча.

3 Отображение списка последних матчей;

– отображение списка определенной длины с краткой информацией о матче (результат противостояния, герой на котором играл пользователь, общий счет, продолжительность матча и т.д.);

– возможность фильтрации матчей по герою, на котором играл пользователь;

– разделение всех матчей на страницы с возможностью переключения между ними;

– переход к анализу определенного матча.

4 Отображение общей информации игрока:

– отображение имени пользователя, уникального идентификатора;

– выпадающее меню при нажатии на компонент;

– возможность перехода в аккаунт Steam;

– возможность выхода из аккаунта.

5 Отображение общего количества за все время и среднего за матч количества:

– убийств;

– смертей;

– помощи;

– золота;

– опыта;

– добитых крипов;

– неотданных крипов;

– урона по вышкам;

– лечения героев.

6 Отображение уровня игры за последние 10 матчей на различных героях, которое состоит из следующих компонент:

– отображение круговой диаграммы, показывающей отношение игр, сыгранных на одном герое. При наведении на ячейку всплывает подробная статистика об уровне игры на данном герое (среднее KDA, GPM, XPM, уровень игры и т.д.);

– отображение аналогичной подробной статистики, только в виде списка.

7 Отображение информации и анализа матча, которое состоит из следующих частей:

– анализ выбора героя. Отображение общего процента побед героя и процента побед героя пользователя в комбинации с союзными и против вражеских героев, а также сгенерированных анализа и совета от ИИ, которому переданы все данные.;

– анализ ранней стадии игры. Отображение статистики убийств, смертей, помощи, заработанного золота и количества добитых крипов в сравнении с игроками аналогичного рейтинга на этом же герое, а также сгенерированных анализа и совета от ИИ, которому переданы все данные;

– анализ сборки предметов. Отображение приобретенных предметов в виде хронологической линии и их подробных описаний, а также сгенерированных анализа и совета от ИИ, которому переданы все данные;

– общая информация о матче. Отображение общего счета, продолжительности матча, всех героев и их сборок и т.д.

# 3 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА

## 3.1 Проектирование и разработка серверного приложения

В данном разделе подробно описаны архитектура и ключевые сервисные модули, реализующие основную бизнес-логику серверного приложения. Данные модули обеспечивают взаимодействие с базой данных MongoDB, внешними API (OpenDota, Steam и т.д.) и обработку бизнес-процессов. Общая схема взаимодействия компонентов серверного приложения представлена на чертеже «Диаграмма компонентов серверной части программного средства» (см. перечень графического материала).

### **3.1.1** Архитектура серверного приложения

Для серверной части ПС была выбрана трехслойная монолитная архитектура. Трехслойная монолитная архитектура – это распространенный архитектурный шаблон для распределенных приложений, который разделяет функциональность системы на три логически и, зачастую, физически отдельные слоя (уровня). Каждый слой имеет свою четко определенную ответственность и взаимодействует только с соседними слоями, что способствует модульности, масштабируемости и облегчает обслуживание системы. Пример схемы данной архитектуры представлен на рисунке 3.1.1.

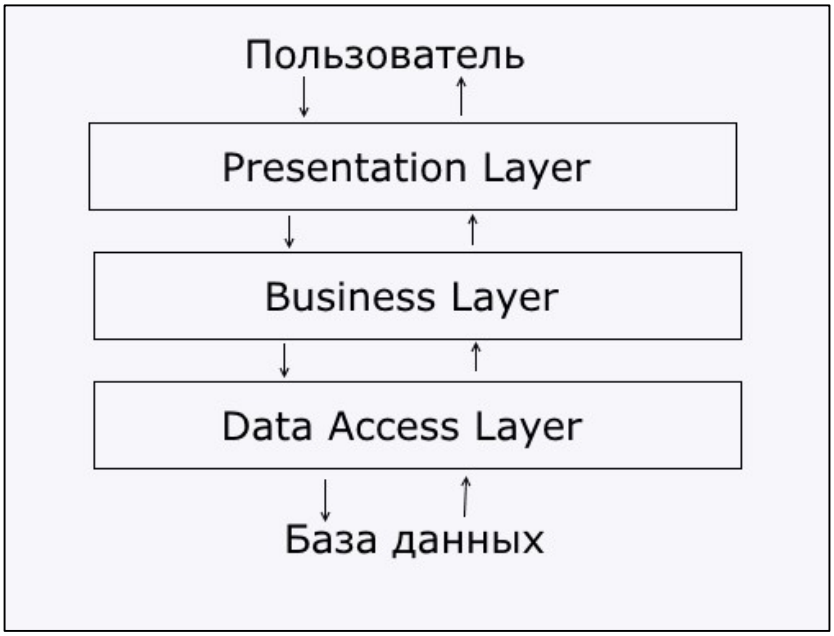



Рисунок 3.1.1 – «Трехслойный» шаблон монолитной архитектуры

Традиционно трехслойная архитектура состоит из следующих слоев:

1 Уровень представления (Presentation Layer / UI Layer). Отвечает за взаимодействие с пользователем, отображение информации и сбор пользовательского ввода. Это может быть Web-API, веб-интерфейс (браузер), настольное приложение, мобильное приложение и т.д.

2 Уровень бизнес-логики / приложения (Business Logic Layer / Application Layer). Содержит основную бизнес-логику приложения, правила, процессы и операции. Он обрабатывает запросы от уровня представления, координирует операции с уровнем данных и возвращает результаты. Этот слой независим от представления и данных. Общается с уровнем представления (принимает запросы) и с уровнем доступа к данным (отправляет запросы на получение/сохранение данных).

3 Уровень доступа к данным (Data Access Layer / Persistence Layer). Отвечает за взаимодействие с базой данных (или другими источниками данных). Этот слой абстрагирует детали работы с внешними API, СУБД, предоставляя унифицированный интерфейс для уровня бизнес-логики. Он выполняет операции CRUD (Create, Read, Update, Delete). Получает запросы от уровня бизнес-логики и взаимодействует непосредственно с базой данных [16].

### **3.1.2** Уровень доступа к данным

Этот слой отвечает за взаимодействие с внешними сервисами (Steam, Stratz, OpenDota, OpenRouter) и внутренней базой данных (MongoDB), инкапсулируя логику получения, агрегации, преобразования и сохранения данных. Общие функции модулей этого слоя включают: загрузку API-ключей из конфигурации приложения, реализацию методов для получения/сохранения данных, взаимодействие с внешними API через HTTP/GraphQL-запросы, преобразование полученных данных во внутренние сущности приложения и обработку ошибок при отсутствии данных или некорректных ответах сервисов.

Основные модули данного слоя:

1 Модуль ProfileRepository:

– обеспечивает централизованный доступ к пользовательским данным из Steam, Stratz и OpenDota;

– реализует методы для получения профиля Steam, базовой информации, рекордов, статистики по героям и информации о матчах;

– использует HeroRepository для сопоставления идентификаторов героев.

2 Модуль ItemRepository:

– обеспечивает централизованный доступ к данным об игровых предметах из Stratz;

– реализует методы для получения полного списка предметов с характеристиками, описаниями и изображениями;

– формирует корректные ссылки на иконки предметов.

3 Модуль HeroRepository:

– обеспечивает централизованный доступ к данным о героях из Stratz и OpenDota;

– реализует методы для получения списков героев через GraphQL Stratz и REST OpenDota.

4 Модуль MatchRepository:

– обеспечивает централизованный доступ к данным о матчах, их статистике и аналитике, используя внутреннюю базу данных (MongoDB) и внешние сервисы Stratz, OpenDota;

– реализует методы для создания и получения матчей из внутренней базы данных;

– взаимодействует с GraphQL API Stratz для получения расширенной статистики и аналитики пиков;

– реализует валидацию входных данных.

5 Модуль AiRepository:

– обеспечивает централизованный доступ к внешнему AI-сервису (OpenRouter);

– реализует методы для отправки запросов и получения ответов от AI через HTTP API для генерации текстовых анализов и советов;

– формирует корректные запросы с необходимыми параметрами.

### **3.1.3** Уровень бизнес-логики

Этот слой отвечает за обработку бизнес-правил, агрегацию данных из репозиториев и подготовку их для уровня представления, а также за взаимодействие с ИИ для генерации персонализированных рекомендаций. Общие функции модулей этого слоя включают: использование соответствующих репозиториев для получения данных, преобразование полученных сущностей в модели, используемые на уровне приложения, и предоставление методов для работы с данными в бизнес-логике.

Основные модули данного слоя:

1 Модуль ProfileService:

– обеспечивает централизованный доступ к пользовательским данным и статистике;

– использует ProfileRepository для получения данных;

– предоставляет методы для получения профиля Steam, базовой информации, рекордов, статистики по героям и последних матчей.

2 Модуль MatchService:

– обеспечивает централизованный доступ к информации о матчах, их аналитике и рекомендациям. Схема алгоритма получения анализа матча представлена на чертеже «Схема алгоритма получения анализа матчей» (см. перечень графического материала);

– использует MatchRepository, ItemRepository, HeroRepository и AiRepository для агрегации данных и аналитики;

– реализует методы для получения полной информации о матче (статистика, лайнинг, пики, предметы);

– формирует запросы к AI-сервису для генерации советов по пикам, ранней стадии и предметам;

– сохраняет обработанную информацию о матче.

3 Модуль HeroService:

– обеспечивает централизованный доступ к информации о героях;

– использует HeroRepository для получения данных;

– предоставляет методы для получения списка героев из Stratz и OpenDota API.

4 Модуль ItemService:

– обеспечивает централизованный доступ к информации о предметах;

– использует ItemRepository для получения данных;

– предоставляет методы для получения списка всех предметов.

### **3.1.4** Уровень представления

Этот слой представляет собой HTTP-интерфейс (API) для клиентских приложений, обрабатывая входящие запросы, извлекая необходимые данные и передавая их на уровень бизнес-логики. Общие функции контроллеров этого слоя включают: реализацию HTTP-маршрутов, использование соответствующих сервисов для обработки бизнес-логики, извлечение идентификаторов пользователей из токенов авторизации, логирование обращений к эндпоинтам и обеспечение защиты эндпоинтов с помощью механизмов авторизации.

Основные модули данного слоя:

1 Контроллер ProfileController:

– обеспечивает HTTP-интерфейс для получения пользовательских данных и статистики;

– реализует маршруты для профиля пользователя, базовой информации, рекордов, статистики по героям и списка последних матчей;

– использует ProfileService.

2 Контроллер MatchesController:

– обеспечивает HTTP-интерфейс для получения детальной информации и аналитики по матчам;

– реализует защищённый маршрут для получения полной информации о конкретном матче;

– использует MatchService.

3 Контроллер HeroesController:

– обеспечивает HTTP-интерфейс для получения информации о героях;

– реализует маршруты для получения списка всех героев и отдельного списка героев из OpenDota;

– использует HeroService.

4 Контроллер ItemsController:

– обеспечивает HTTP-интерфейс для получения информации о предметах;

– реализует маршрут для получения списка всех предметов;

– использует ItemService.

5 Контроллер AuthController:

– обеспечивает HTTP-интерфейс для аутентификации и управления сессией пользователя через Steam;

– реализует маршруты для входа и выхода пользователя;

– предоставляет маршрут для получения информации о текущем пользователе (Steam ID);

– использует стандартные механизмы аутентификации ASP.NET и обрабатывает ошибки авторизации.

## 3.2 Проектирование и разработка клиентского приложения

В данном разделе подробно описана функциональность клиентского приложения.

### **3.2.1** Архитектура клиентского приложения

Клиентское приложение разработано в соответствии с принципами Feature-Sliced Design (FSD), что обеспечивает модульность,

масштабируемость и высокую поддерживаемость кода. Такой подход позволяет четко разделять ответственность между слоями, ускоряет внедрение новых функций и облегчает командную работу над проектом.

FSD – это архитектурный шаблон для фронтенд-разработки, который упорядочивает код вокруг конкретных функций продукта. Его ключевая идея – декомпозиция приложения на изолированные, самодостаточные «слайсы» (срезы), которые выстроены по принципу слоёв и сегментов. Такой подход с жёсткими правилами зависимостей между частями кода способствует высокой согласованности внутри модулей и минимальной связанности между ними (рисунок 3.2.1) [17].

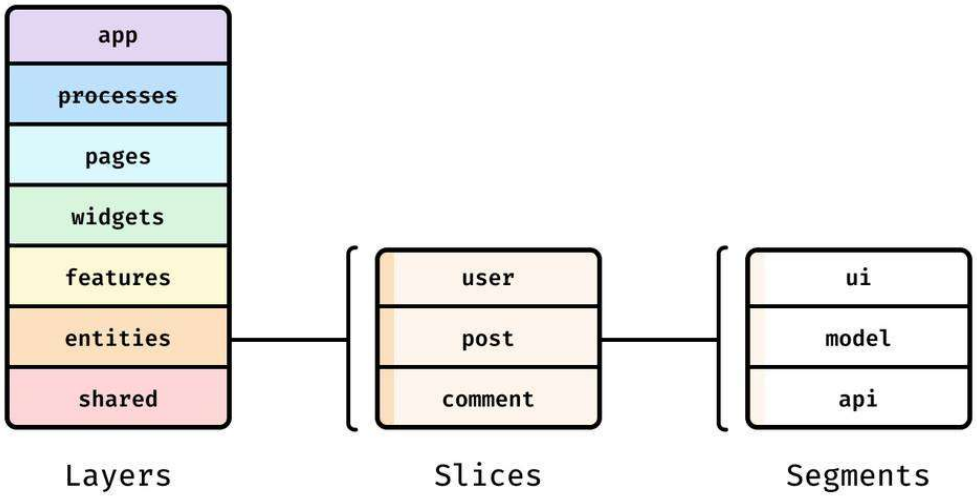



Рисунок 3.2.1 – Схема FSD

Архитектура FSD предусматривает стандартизированные слои, каждый из которых имеет свою чёткую ответственность и ориентирован на определённый аспект бизнес-логики или инфраструктуры. Слои организованы снизу вверх по зависимостям: каждый слой может использовать только те слои, что находятся ниже него.

Слои в FSD:

1 App (Application). Это корневой слой, отвечающий за инициализацию всего приложения. Здесь настраиваются глобальные конфигурации, маршрутизация, провайдеры состояния и применяются общие стили. Он служит единой точкой входа для запуска приложения.

2 Pages (Страницы). Этот слой содержит полные представления пользовательского интерфейса, которые напрямую связаны с маршрутами

верхнего уровня приложения. Каждая страница собирает необходимые виджеты и фичи для формирования целостного интерфейса.

3 Widgets (Виджеты). Представляют собой самодостаточные, крупные UI-компоненты, которые комбинируют несколько функций или сущностей. Они предназначены для повторного использования на различных страницах (например, шапка сайта, боковая панель с навигацией или блок с общей статистикой).

4 Features (Функции). Здесь реализуется конкретная интерактивная функциональность, которая предоставляет прямую бизнес-ценность пользователю. Например, модуль авторизации пользователя, система фильтрации матчей, форма для отправки отзыва или функция добавления в избранное.

5 Entities (Сущности). Этот слой содержит определения основных бизнес-объектов приложения, таких как «Пользователь», «Матч», «Герой» или «Предмет». Он включает в себя их модели данных, связанную с ними логику, а также механизмы для взаимодействия с серверным API.

6 Shared (Общие). Это базовый слой, включающий универсальные, переиспользуемые компоненты и утилиты, которые не зависят от специфической бизнес-логики. К ним относятся UI-набор (базовые компоненты интерфейса), общие вспомогательные функции, конфигурации сетевых запросов (например, Axios) и глобальные константы приложения.

Такая структура FSD обеспечивает высокую степень организации и предсказуемости кода, что упрощает масштабирование проекта, совместную разработку и последующее обслуживание.

### **3.2.2** App Layer (Слой приложения)

Этот слой отвечает за инициализацию приложения, глобальные провайдеры, маршрутизацию и конфигурацию. Здесь происходит подключение глобальных стилей, цветовых тем, настройка Redux Store, а также определение маршрутов для навигации между страницами.

### **3.2.3** Pages Layer (Слой страниц)

Страницы – это самостоятельные сценарии использования, которые агрегируют необходимые виджеты, фичи и сущности. Каждая страница отвечает за отображение определенного пользовательского сценария:

1 Профиль пользователя. Отображает карточку профиля, ранг, рекорды и анализ эффективности.

2 История матчей. Предоставляет фильтрацию и просмотр сыгранных матчей.

3 Анализ матча. Детализированная аналитика по отдельному матчу.

4 Welcome-страница. Приветственный экран с каруселью и кнопкой входа.

**3.2.4** Widgets Layer (Слой виджетов)

Виджеты – это крупные независимые компоненты, которые могут использоваться на различных страницах. Виджеты использованные в клиентском приложении:

– Анимированный лоадер;

– Логотип приложения;

– Пагинация для списков;

– Карточка профиля Steam для хедера.

**3.2.5** Features Layer (Слой функций)

Фичи инкапсулируют бизнес-логику, UI и стор, реализуя отдельные пользовательские сценарии. Примеры:

– Селектор героев с загрузкой из OpenDota;

– Анализ матча (пики, лайнинг, предметы);

– История матчей пользователя;

– Анализ эффективности по героям;

– Заголовок профиля с аватаром и именем;

– Карточка ранга пользователя;

– Рекорды и статистика.

**3.2.6** Entities Layer (Слой сущностей)

Содержит модели и сторы для базовых сущностей приложения, например:

– Герой;

– Предмет;

– Профиль пользователя;

– Матч.

**3.2.7** Shared Layer (Общий слой)

Включает переиспользуемые компоненты, утилиты, базовые модели и API-запросы:

– Общие UI-компоненты (карточки, диалоги);

– Утилиты и вспомогательные функции;
– Типы и модели данных;
– Базовые запросы к API.

### 3.3 Развертывание приложения

Для развертывания приложения используется технология контейнеризации Docker и инструмент оркестрации Docker Compose, что обеспечивает простоту запуска и масштабируемость всех компонентов системы. Схема развертывания приложения представлена на чертеже «Схема развертывания программного средства» (см. перечень графического материала).

Приложение состоит из трёх основных сервисов:

1 MongoDB – база данных для хранения информации.

2 Backend – серверная часть на ASP.NET Core, реализующая бизнес-логику и API.

3 Frontend – клиентская часть на основе современного JavaScript-фреймворка.

В файле docker-compose.yml и в файлах окружения каждого сервиса задаются необходимые переменные, такие как адреса API, параметры подключения к базе данных и другие настройки.

Использование Docker и Docker Compose позволяет быстро развернуть приложение на любой машине с установленным Docker, изолировать окружения сервисов, упростить масштабирование и обновление компонентов системы.

Серверная часть, реализованная на ASP.NET Core, использует трехслойную монолитную архитектуру. Это обеспечивает четкое разделение логики: уровень доступа к данным взаимодействует с MongoDB и внешними API (Steam, Stratz, OpenDota), уровень бизнес-логики обрабатывает правила и интегрируется с ИИ-сервисом, а уровень представления предоставляет защищенные HTTP-эндпоинты.

Клиентское приложение разработано на основе Feature-Sliced Design (FSD). Этот подход структурирует код по слоям, что значительно повышает модульность, поддерживаемость и масштабируемость.

Для развертывания всего решения используются Docker и Docker Compose, обеспечивающие простоту запуска, изоляцию сервисов (MongoDB, Backend, Frontend) и масштабируемость.

В итоге, выбранные архитектура и технологии создают надежную, гибкую и легко масштабируемую платформу, способную эффективно обрабатывать данные, предоставлять аналитику и развиваться в будущем.

# 4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

Тестирование – фундаментальный этап в разработке веб-приложений. Это не просто часть процесса, а тщательно спланированная и организованная деятельность, закладывающая основу для каждого успешного проекта. Оно включает в себя планирование, проведение тестов, внедрение и дальнейшее сопровождение. Обеспечение качества приложения критически важно для создания высококачественных продуктов, которые всегда соответствуют ожиданиям пользователей.

## 4.1 Ручное тестирование программного средства

Ручное тестирование – это метод проверки программного обеспечения, при котором тестировщик вручную выполняет проверки, не используя автоматизированные средства. Его главная задача – тщательно находить ошибки, дефекты и проблемы в программах, продуктах и системах. Несмотря на свою примитивность, этот метод крайне важен для выявления критических неполадок, связанных с функциональностью, удобством использования и общей производительностью. В конечном итоге, ручное тестирование гарантирует, что приложение соответствует всем требованиям и будет работать без сбоев.

Таблица 4.1.1 – Результаты тестирования программного средства

| Предусловие | Описание теста | Ожидаемый результат | Статус теста |
|---|---|---|---|
| 1 | 2 | 3 | 4 |
| Пользователь открыл клиент. | 1 нажать кнопку авторизации.<br>2 Войти через свой аккаунт Steam. | 1 Происходит процесс авторизации.<br>2 Пользователь перенаправляется на главную страницу своего профиля. | пройден |

Продолжение таблицы 4.1.1

| 1 | 2 | 3 | 4 |
|---|---|---|---|
| Пользователь открыл клиент. | 1 Ввести в адресную строку адрес страницы профиля профилю. | После загрузки пользователь перенаправляется на страницу авторизации через Steam. | пройден |
| Авторизованный пользователь открыл клиент. | 1 Открыть страницу профиля пользователя.<br>2 Нажать на карточку с общей информацией.<br>3 Нажать кнопку выхода. | Пользователя перенаправляет на страницу входа. | пройден |
| Авторизованный пользователь открыл клиент. | Нажать кнопку перехода в аккаунт Steam. | Пользователя перенаправляет в аккаунт Steam в новой вкладке. | пройден |
| Авторизованный пользователь открыл клиент. | 1 Перейти на страницу матчей.<br>2 Перейти на следующую страницу с помощью кнопки «Вперед».<br>3 Перейти на предыдущую страницу с помощью кнопки «Назад». | 1 Отображается список последних матчей с информацией о каждом матче.<br>2 Отображается следующая страница матчей, и кнопка «Назад» становится доступной.<br>3 Отображается прошлая страница матчей, и кнопка «Назад» становится недоступной. | пройден |

Продолжение таблицы 4.1.1

| 1 | 2 | 3 | 4 |
|---|---|---|---|
| Авторизованный пользователь открыл клиент на странице с матчами. | 1 Открыть селектор с фильтром героя.<br>2 Выбрать героя. | Показываются только матчи с игрой пользователя на выбранном герое. | пройден |
| Авторизованный пользователь открыл клиент на странице с матчами. | Нажать кнопку «Анализ матча» | После показа загрузочного экрана, пользователь перенаправляется на страницу с полноценным анализом матча. | |

## 4.2 Тестирование компонентов пользовательского интерфейса

В рамках разработки проекта были написаны модульные тесты для компонентов пользовательского интерфейса с использованием библиотеки Jest и React Testing Library. Эти тесты направлены на проверку корректности рендеринга компонентов и их базовой функциональности.

Jest – это быстрый и современный тестовый фреймворк для JavaScript, который обеспечивает удобный синтаксис для написания тестов и прекрасно интегрируется с React.

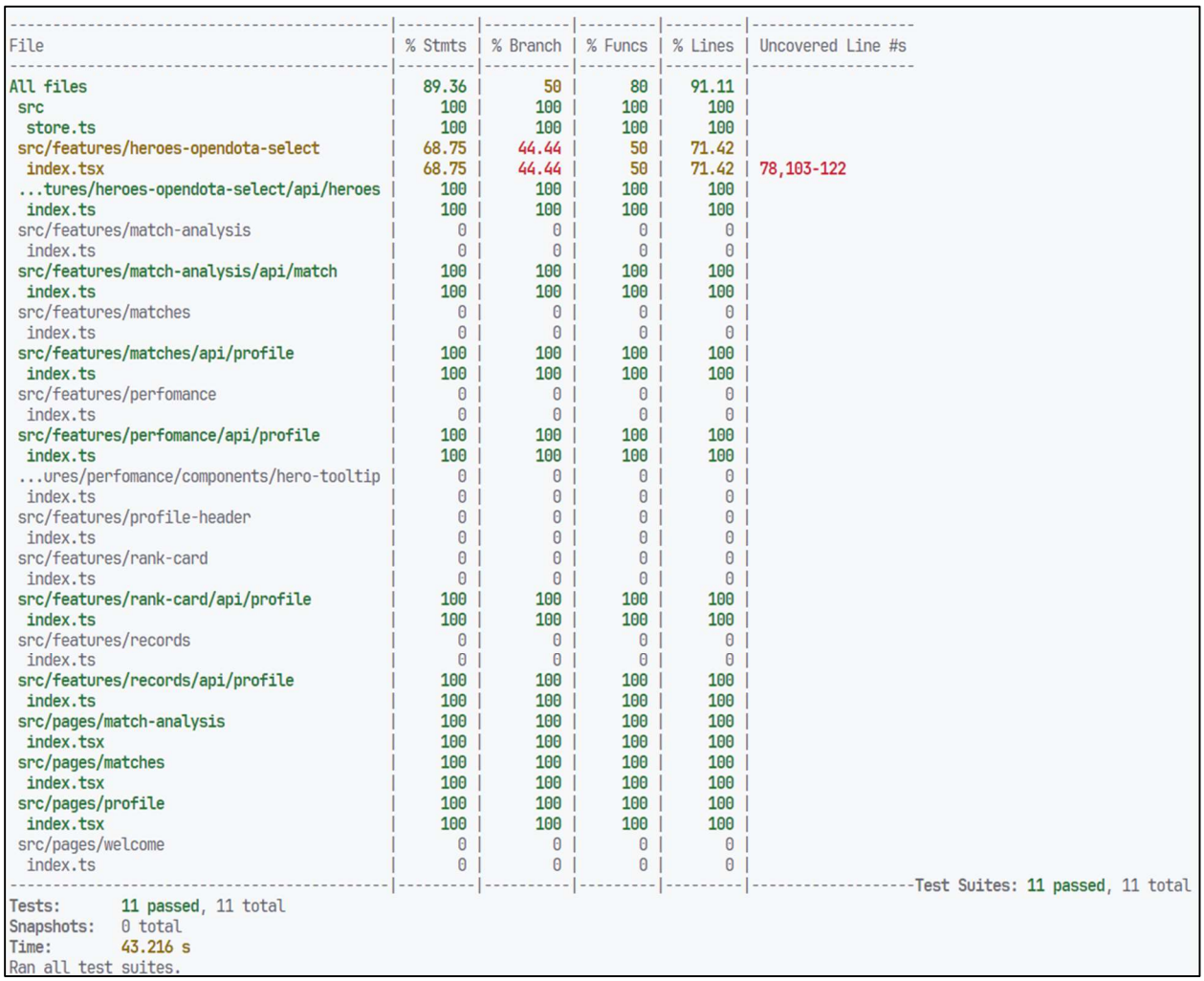
React Testing Library – это библиотека, которая позволяет тестировать компоненты React с точки зрения пользователя, фокусируясь на доступности и взаимодействии с DOM.

Smoke-тесты проверяют, что компонент корректно отображается без ошибок. Например, тесты для компонентов Label, Separator, Skeleton и Toaster (из библиотеки Sonner) подтверждают их отображение в DOM.

Тесты для таких компонентов, как Tabs, проверяют, что компонент корректно отображает переданные ему данные (например, вкладки и их содержимое). В некоторых случаях, например, для компонента Tabs, тесты также проверяют взаимодействие пользователя с компонентом (например, возможность переключения между вкладками).

Для оценки качества и полноты тестирования в проекте использовался инструмент покрытия кода (рисунок 4.2.1). Покрытие кода – это метрика,

которая показывает, какая часть кода была выполнена во время запуска тестов. Это позволяет выявить непокрытые участки кода и определить, какие части приложения требуют дополнительной проверки.



| File | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s |
|---|---|---|---|---|---|
| All files | 89.36 | 50 | 80 | 91.11 | |
| src | 100 | 100 | 100 | 100 | |
| store.ts | 100 | 100 | 100 | 100 | |
| src/features/heroes-opendota-select | 68.75 | 44.44 | 50 | 71.42 | |
| index.tsx | 68.75 | 44.44 | 50 | 71.42 | 78,103-122 |
| ...tures/heroes-opendota-select/api/heroes | 100 | 100 | 100 | 100 | |
| index.ts | 100 | 100 | 100 | 100 | |
| src/features/match-analysis | 0 | 0 | 0 | 0 | |
| index.ts | 0 | 0 | 0 | 0 | |
| src/features/match-analysis/api/match | 100 | 100 | 100 | 100 | |
| index.ts | 100 | 100 | 100 | 100 | |
| src/features/matches | 0 | 0 | 0 | 0 | |
| index.ts | 0 | 0 | 0 | 0 | |
| src/features/matches/api/profile | 100 | 100 | 100 | 100 | |
| index.ts | 100 | 100 | 100 | 100 | |
| src/features/perfomance | 0 | 0 | 0 | 0 | |
| index.ts | 0 | 0 | 0 | 0 | |
| src/features/perfomance/api/profile | 100 | 100 | 100 | 100 | |
| index.ts | 100 | 100 | 100 | 100 | |
| ...ures/perfomance/components/hero-tooltip | 0 | 0 | 0 | 0 | |
| index.ts | 0 | 0 | 0 | 0 | |
| src/features/profile-header | 0 | 0 | 0 | 0 | |
| index.ts | 0 | 0 | 0 | 0 | |
| src/features/rank-card | 0 | 0 | 0 | 0 | |
| index.ts | 0 | 0 | 0 | 0 | |
| src/features/rank-card/api/profile | 100 | 100 | 100 | 100 | |
| index.ts | 100 | 100 | 100 | 100 | |
| src/features/records | 0 | 0 | 0 | 0 | |
| index.ts | 0 | 0 | 0 | 0 | |
| src/features/records/api/profile | 100 | 100 | 100 | 100 | |
| index.ts | 100 | 100 | 100 | 100 | |
| src/pages/match-analysis | 100 | 100 | 100 | 100 | |
| index.tsx | 100 | 100 | 100 | 100 | |
| src/pages/matches | 100 | 100 | 100 | 100 | |
| index.tsx | 100 | 100 | 100 | 100 | |
| src/pages/profile | 100 | 100 | 100 | 100 | |
| index.tsx | 100 | 100 | 100 | 100 | |
| src/pages/welcome | 0 | 0 | 0 | 0 | |
| index.ts | 0 | 0 | 0 | 0 | |

Test Suites: 11 passed, 11 total

```
Tests:       11 passed, 11 total
Snapshots:   0 total
Time:        43.216 s
Ran all test suites.
```

Рисунок 4.2.1 – Отчет о покрытии тестами компонентов

При разработке проекта, модульные тесты для компонентов пользовательского интерфейса были успешно созданы. Благодаря комбинации Jest и React Testing Library, удалось написать надёжные и эффективные тесты, обеспечивающие корректность отображения и базовую функциональность компонентов.

# 5 МЕТОДИКА ИСПОЛЬЗОВАНИЯ РАЗРАБОТАННОГО ПРОГРАММНОГО СРЕДСТВА

Для использования приложения необходимо:

1 Открыть браузер на любом устройстве.

2 Ввести в адресную строку адрес сайта.

Клиентское приложение откроется прямо в браузере.

Если пользователь не авторизовывался в приложении ранее, то он увидит приветственную страницу (рисунок 5.1).

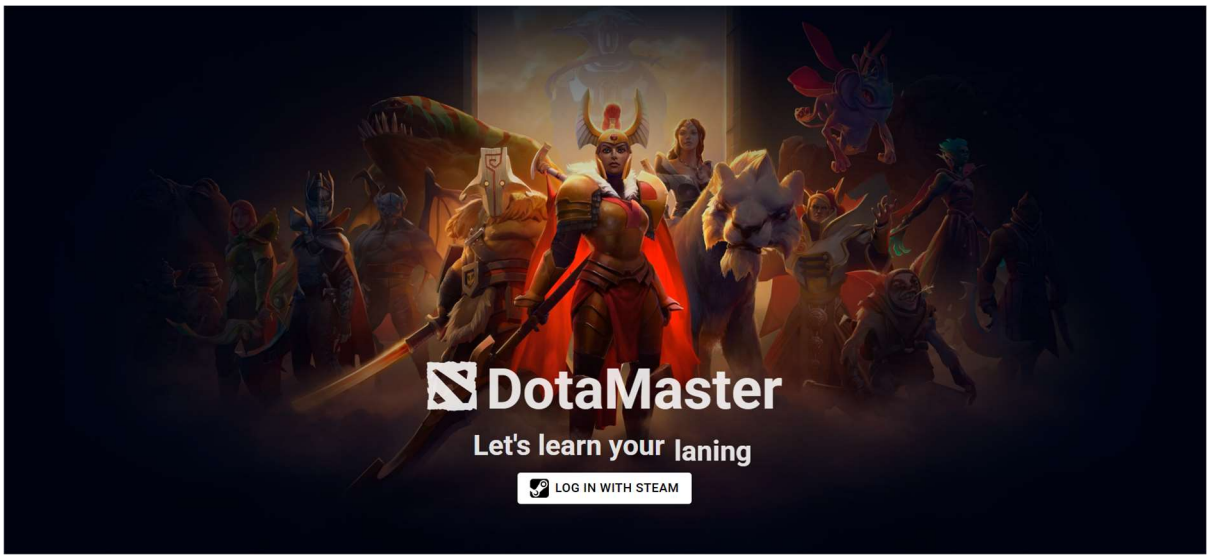



Рисунок 5.1 – Начальная страница приложения

После нажатия кнопки авторизации, пользователь будет перенаправлен на страницу входа официальный сайта Steam (рисунок 5.2).

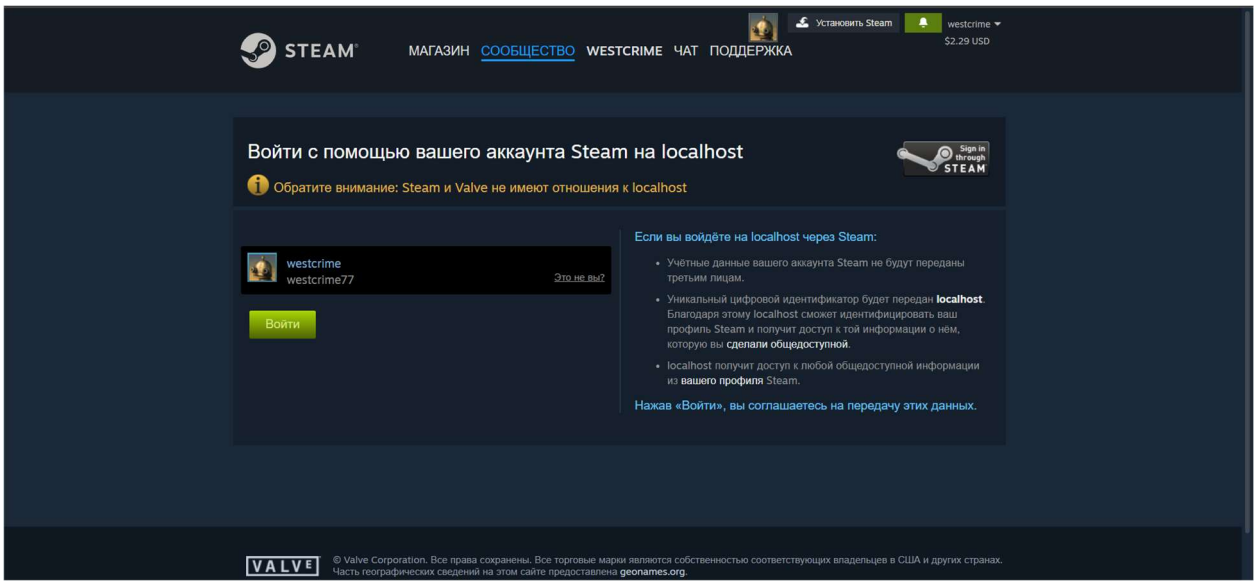



Рисунок 5.2 – Страница входа через аккаунт Steam

После успешной авторизации пользователь перенаправляется на свой профиль с статистикой, общей информацией и т.д. (рисунок 5.3).

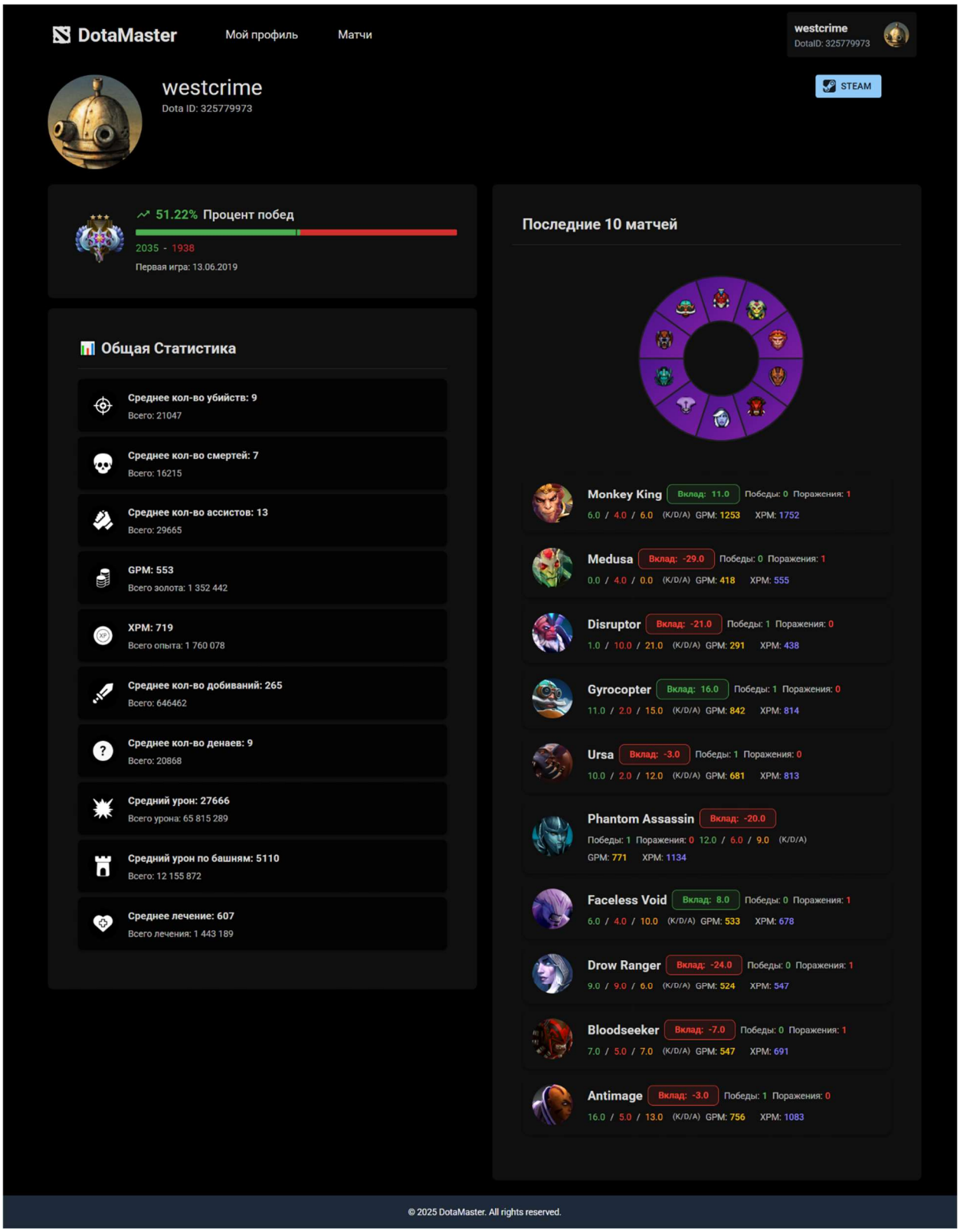



Рисунок 5.3 – Страница профиля

Есть возможность перейти в аккаунт Steam, нажав на кнопку с соответствующим значком. Чтобы выйти из профиля требуется нажать на карточку с информацией о Steam аккаунте (рисунок 5.4).

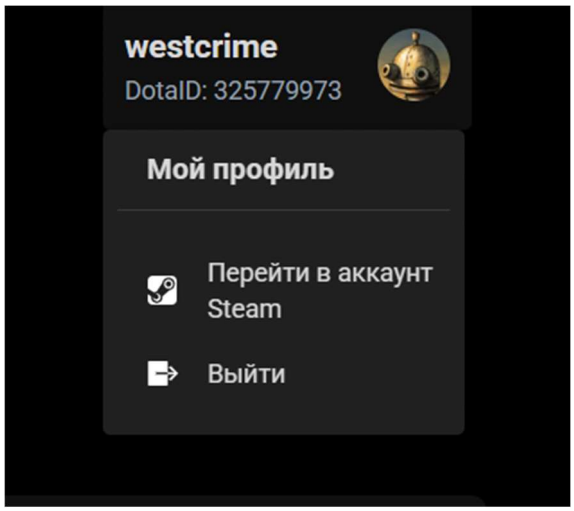



Рисунок 5.4 – Меню действий аккаунта

Чтобы перейти на страницу матчей, нужно нажать на соответствующую вкладку и после загрузчика, появляется страница с последними матчами (рисунок 5.5).

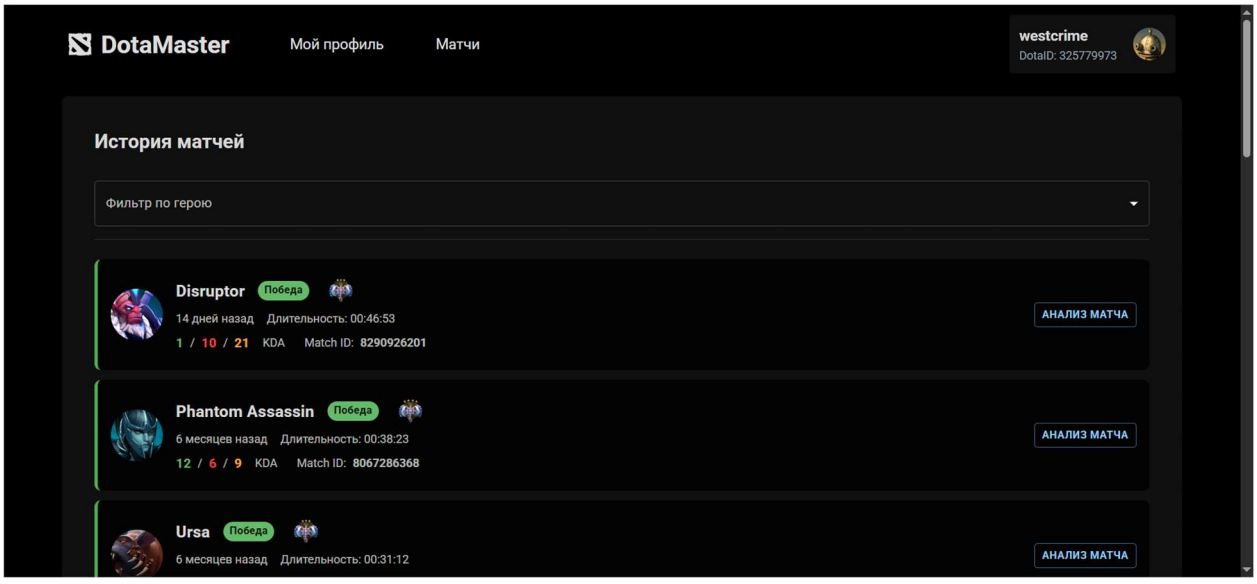



Рисунок 5.5 – Страница с матчами

Есть возможность применить фильтрацию по выбранному герою. Для этого требуется нажать на селектор «Фильтр по герою» и выбрать нужного героя (рисунок 5.6).

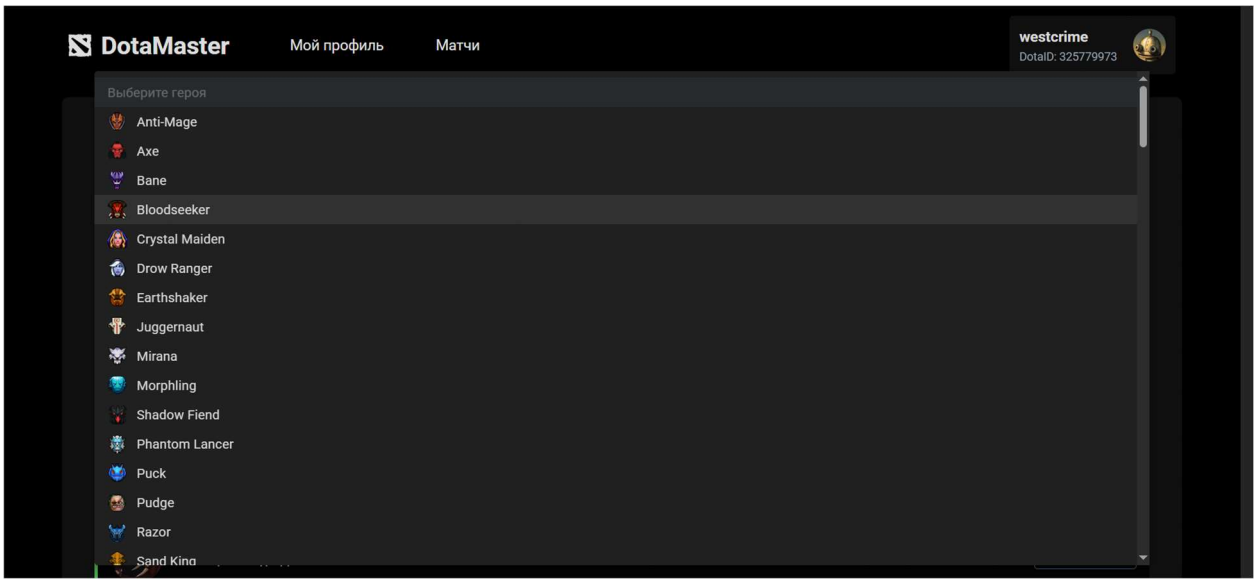



Рисунок 5.6 – Страница с матчами

Также если требуется просмотреть больше последних матчей, реализована постраничная навигация с помощью кнопок «Назад» и «Вперед» (рисунок 5.7).




Рисунок 5.7 – Постраничная навигация

Чтобы проанализировать какой-либо матч, требуется нажать на соответствующую кнопку «Анализ матча». После нажатия на кнопку и окончания процесса анализа пользователя перенаправляет на страницу с анализом, включая анализ выбора героев (рисунок 5.8), анализ лайнинга (рисунок 5.9), анализ сборки предметов (рисунок 5.10) и общая информация о матче (рисунок 5.11).

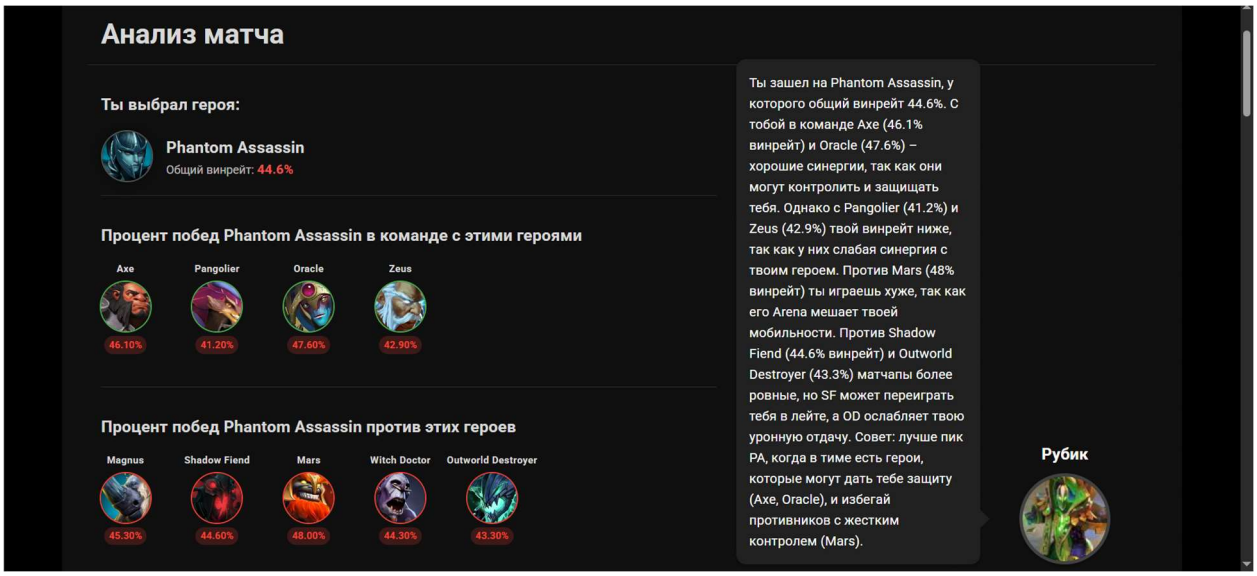



Рисунок 5.8 – Анализ выбора героев

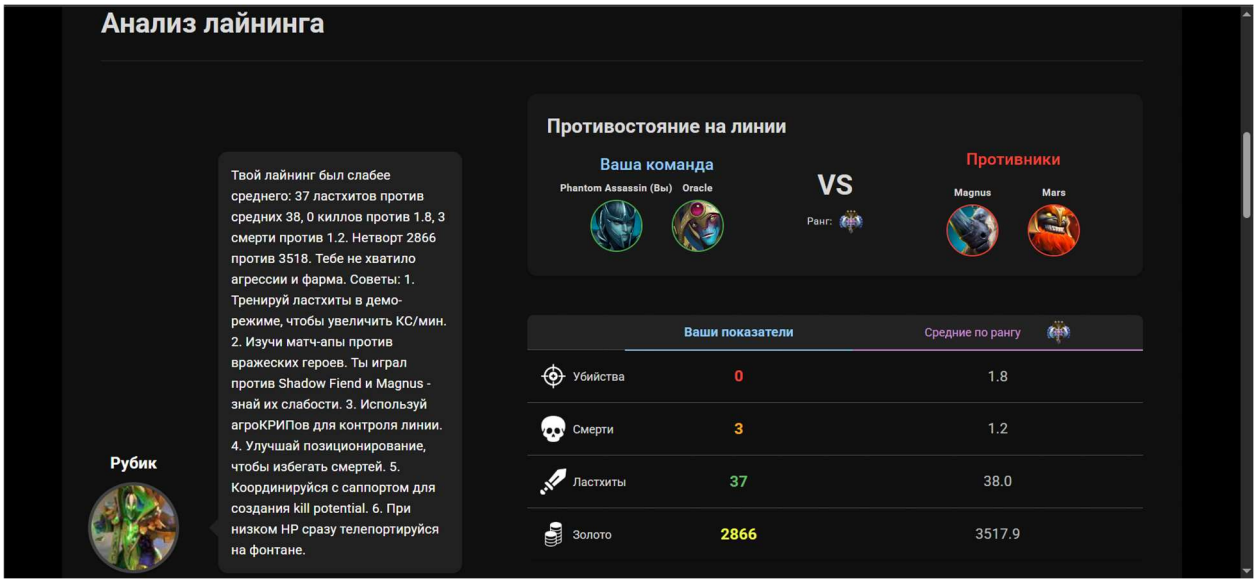



Рисунок 5.9 – Анализ лайнинга

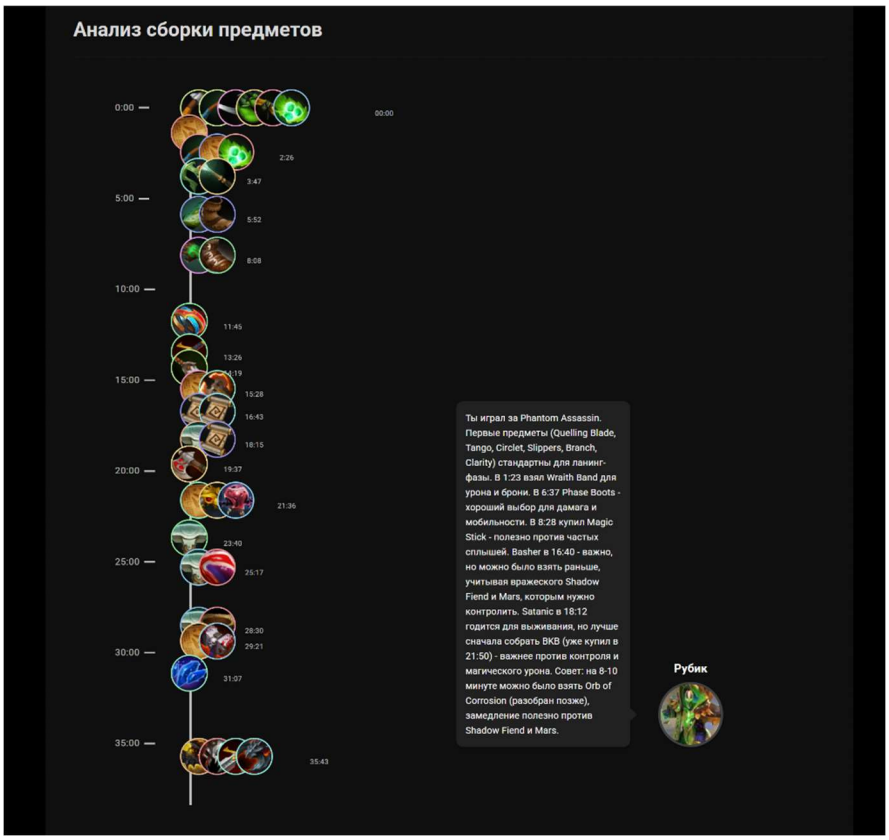



Рисунок 5.10 – Анализ сборки предметов

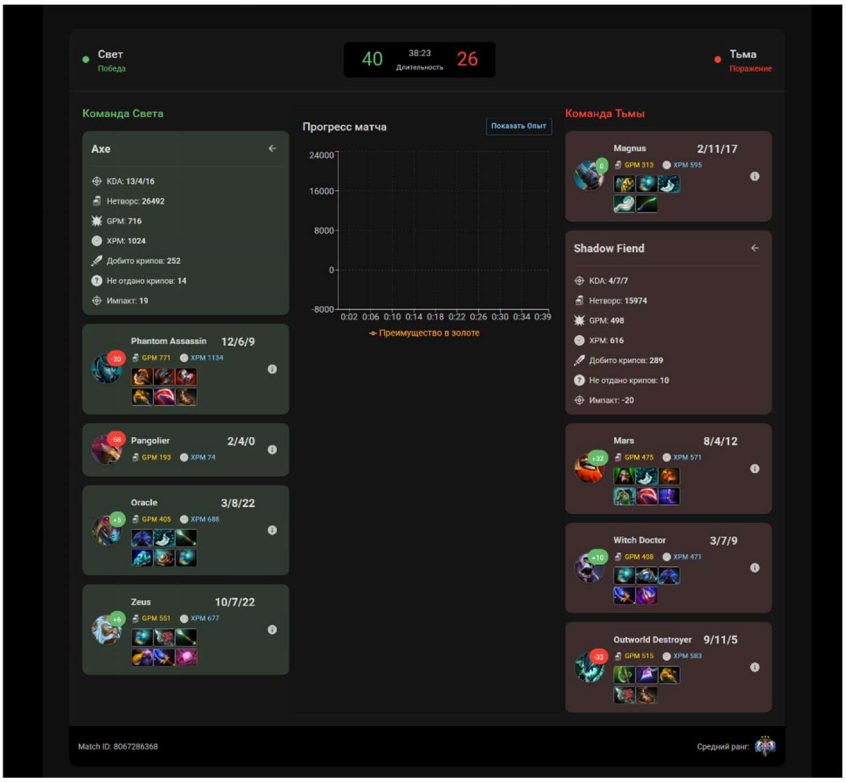



Рисунок 5.11 – Общая информация о матче

# 6 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ И РЕАЛИЗАЦИИ НА РЫНКЕ ПЛАТФОРМЫ ДЛЯ АНАЛИЗА СТАТИСТИКИ ИГРОКА В КОМПЬЮТЕРНОЙ ИГРЕ DOTA2

## 6.1 Характеристика программного средства, разрабатываемого для реализации на рынке

Целью разработки является создание специализированной платформы для анализа статистики игроков в DOTA 2, которая предоставляет пользователям удобный инструмент для глубокого анализа их игровых данных. Платформа использует открытые данные из публичных API связанных с данной игрой, визуализирует их в понятном виде (графики, схемы, таблицы) и предлагает персонализированные рекомендации для улучшения навыков игрока на основе анализа матчей с использованием искусственного интеллекта.

Приложение будет иметь интуитивно понятный интерфейс, позволяющий пользователям быстро ориентироваться и находить нужные функции. Оно предоставит возможность авторизации через игровую платформу Steam. Просмотр различных аспектов игрока, выбор промежутка времени анализа, вариативное представление данных. Возможность детального анализа отдельных матчей (таймлайн событий, ключевые моменты, эффективность действий). Сравнение статистики игрока с усредненными показателями других игроков аналогичного уровня. Генерация персонализированных советов для улучшения навыков (например, улучшение фарма, выбор героев, стратегии для конкретных ролей).

Ограниченный доступ к данной платформе можно получить бесплатно для анализа 3 матчей в целях завлечения пользователей, далее доступ будет предоставляться по ежемесячной платной подписке.

Игровая индустрия является одной из самых динамично развивающихся отраслей. Этот рынок показывает устойчивый и активный рост, и на нем уже присутствует несколько аналогов описанной выше платформы.

1 Stratz API. Stratz предоставляет доступ к данным матчей и статистике игроков, но не предлагает персонализированных рекомендаций на основе ИИ. Stratz ориентирован на разработчиков и продвинутых пользователей, что делает его менее доступным для новичков.

2 Dotabuff. Dotabuff предоставляет базовую статистику и анализ матчей, но не предлагает персонализированных рекомендаций. Отсутствует интеграция ИИ для анализа слабых мест игрока. Интерфейс Dotabuff ориентирован на общую статистику, а не на индивидуальный анализ.

3 OpenDota. OpenDota предоставляет доступ к данным матчей и API для разработчиков, но не предлагает удобного интерфейса для анализа и рекомендаций. Отсутствует функционал для генерации советов на основе ИИ.

Разрабатываемая платформа предлагает уникальное сочетание глубокого анализа данных, удобного интерфейса и персонализированных рекомендаций на основе ИИ, что выделяет ее среди существующих аналогов. Она ориентирована на широкий круг пользователей, от новичков до профессионалов, и предоставляет инструменты для улучшения навыков и понимания игры. На рынке подобных решений с интеграцией ИИ практически нет, что открывает возможности для занятия уникальной ниши.

## 6.2 Расчёт инвестиций в разработку программного средства

Расчёт затрат на основную заработную плату разработчиков производится исходя из количества людей, которые занимаются разработкой программного продукта, месячной зарплаты каждого участника процесса разработки и сложности выполняемой ими работы. Затраты на основную заработную плату рассчитаны по формуле:

$$З_о = К_{пр} \sum_{i=1}^{n} З_{чi} \cdot t_i, \quad (6.1)$$

где $К_{пр}$ – коэффициент премий и иных стимулирующих выплат;

$n$ – категории исполнителей, занятых разработкой программного средства;

$З_{чi}$ – часовая заработная плата исполнителя $i$-й категории, р;

$t_i$ – трудоемкость работ, выполняемых исполнителем $i$-й категории, ч.

Разработкой серверной части приложения занимается backend-разработчик. Написание графического интерфейса приложения лежит на frontend-разработчике. Задачами backend-разработчика являются создание серверной части с интеграцией искусственного интеллекта. Frontend-

разработчик возьмет на себя разработку графического интерфейса, коммуникации между сервером и веб приложением, обработку пользовательского ввода.

Месячная заработная плата основана на медианных показателях для Junior backend-разработчика за 2024 год по Республике Беларусь, которая составляет примерно 600 Долларов США в месяц, а для Junior frontend-разработчика около 550 Долларов США. По состоянию на 5 марта 2025 года, 1 Доллар США по курсу Национального Банка Республики Беларусь составляет 3,2223 Белорусских рублей.

В перерасчёте на Белорусские рубли месячные оклады для backend-разработчика и frontend-разработчика соответственно составляют составляет 1 772,22 и 1 933,38 Белорусских рублей соответственно.

Часовой оклад исполнителей высчитывается путём деления их месячного оклада на количество рабочих часов в месяце, то есть 168 часов.

Коэффициент премии приравнивается к единице, так как она входит сумму заработной платы. Затраты на основную заработную плату приведены в таблице:

Таблица 6.1 – Затраты на основную заработную плату

| Категория исполнителя | Месячный оклад, р | Часовой оклад, р | Трудоёмкость работ, ч | Итого, р |
|---|---|---|---|---|
| Frontend-разработчик | 1 772,22 | 10,55 | 168 | 1 772,22 |
| Backend-разработчик | 1 933,38 | 11,51 | 168 | 1 933,38 |
| Итого | | | | 3 705,6 |
| Премия и иные стимулирующие выплаты (0%) | | | | 0 |
| Всего затраты на основную заработную плату разработчиков | | | | 3 705,6 |

Расчёт затрат на дополнительную заработную плату команды разработчиков рассчитывается по формуле:

$$З_д = \frac{З_о \cdot Н_д}{100}, \quad (6.2)$$

где $Н_д$ – норматив дополнительной заработной платы.

Значение норматива дополнительной заработной платы принимаем

равным 10 %.

Размер отчислений на социальные нужды определяется согласно ставке отчислений принимаем равным 34%. Расчёт отчислений на социальные нужды вычисляется по формуле:

$$Р_{соц} = \frac{(З_о + З_д) \cdot Н_{соц}}{100}, \quad (6.3)$$

где $Н_{соц}$ – норматив отчислений в ФСЗН.

Расчёт затрат на прочие расходы определяется при помощи норматива прочих расчётов. Эта величина имеет значение 30%. Расчёт прочих расходов вычисляется по формуле:

$$Р_{пр} = \frac{З_о \cdot Н_{пр}}{100}, \quad (6.4)$$

где $Н_{пр}$ – норматив прочих расходов.

Для того, чтобы рассчитать расходы на реализацию, необходимо знать норматив расходов на неё. Принимаем значение норматива равным 3%. Формула, которая использована для расчёта расходов на реализацию:

$$Р_р = \frac{З_о \cdot Н_р}{100}, \quad (6.5)$$

где $Н_р$ – норматив расходов на реализацию.

Определяем общую сумму затрат на разработку и реализацию как сумму ранее вычисленных расходов: на основную заработную плату разработчиков, дополнительную заработную плату разработчиков, отчислений на социальные нужды, расходы на реализацию и прочие расходы. Значение определяется по формуле:

$$З_р = З_о + З_д + Р_{соц} + Р_{пр} + Р_р \quad (6.6)$$

Таким образом, величина затрат на разработку программного средства высчитывается по указанной выше формуле и указана в таблице:

Таблица 6.2 – Затраты на разработку

| Название статьи затрат | Формула/таблица для расчёта | Значение, р. |
|---|---|---|
| 1. Основная заработная плата разработчиков | См. таблицу 1.1 | 3 705,6 |
| 2. Дополнительная заработная плата разработчиков | $З_д = \frac{3\,705{,}60 \cdot 10}{100}$ | 370,05 |
| 3. Отчисление на социальные нужды | $Р_{соц} = \frac{(3\,705{,}60 + 370{,}05) \cdot 34}{100}$ | 1 385,72 |
| 4. Прочие расходы | $Р_{пр} = \frac{3\,705{,}60 \cdot 30}{100}$ | 1 111,68 |
| 5. Расходы на реализацию | $Р_р = \frac{3\,705{,}60 \cdot 3}{100}$ | 111,17 |
| 6. Общая сумма затрат на разработку и реализацию | $З_р = 3\,705{,}6 + 370{,}05 + +1\,385{,}72 + 1\,111{,}68 + 111{,}17$ | 6 684,22 |

### 6.3 Расчёт экономического эффекта от реализации программного средства на рынке

Для расчёта экономического эффекта организации-разработчика программного средства, а именно чистой прибыли, необходимо знать такие параметры как объем продаж, цену реализации и затраты на разработку.

Для оценки экономического эффекта программного продукта, предположим, что около 10 000 человек будут заинтересованы данным приложением. Из них 6 000 человек будут использовать его, а 2 500 приобретут расширенную версию. Цена на расширенную версию приложения составляет 4,99 долларов США. Таким образом, отпускная цена копии программного средства составляет 16,08 Белорусских рубля.

Для расчёта прироста чистой прибыли необходимо учесть налог на добавленную стоимость, который высчитывается по следующей формуле:

$$НДС = \frac{Ц_{отп} \cdot N \cdot Н_{д.с}}{100\% + Н_{д.с}}, \quad (6.7)$$

где N – количество копий(лицензий) программного продукта, реализуемое за год, шт.;

$Ц_{отп}$ – отпускная цена копии программного средства, р.;

N – количество приобретённых лицензий;

$Н_{д.с}$ – ставка налога на добавленную стоимость, %.

Ставку налога на добавленную стоимость по состоянию принимаем равной 20%. Используя данное значение, посчитаем НДС:

$$НДС = \frac{16{,}08 \cdot 2\,500 \cdot 20\%}{100\% + 20\%}, = 6\,700{,}00 \text{ р.}$$

Посчитав налог на добавленную стоимость, можно рассчитать прирост чистой прибыли, которую получит разработчик от продажи программного продукта. Для этого используется формула:

$$\Delta П_{ч}^{р} = (Ц_{отп} \cdot N - НДС) \cdot Р_{пр} \cdot \left(1 - \frac{Н_{п}}{100}\right), \quad (6.8)$$

где *N* – количество копий(лицензий) программного продукта, реализуемое за год, шт.;

$Ц_{отп}$ – отпускная цена копии программного средства, р.;

НДС – сумма налога на добавленную стоимость, р.;

$Н_{п}$ – ставка налога на прибыль, %;

$Р_{пр}$ – рентабельность продаж копий.

Ставку налога на прибыль принимаем равной 20%. Рентабельность продаж копий взята в размере 40%. Зная ставку налога и рентабельность продаж копий (лицензий), рассчитывается прирост чистой прибыли для разработчика:

$$\Delta П_{ч}^{р} = (16{,}08 \cdot 2\,500 - 6\,700) \cdot 40\% \cdot \left(1 - \frac{20}{100}\right) = 10\,720{,}00$$

## 6.4 Расчёт показателей экономической эффективности разработки и реализации программного средства на рынке

Для того, чтобы оценить экономическую эффективность разработки и реализации программного средства на рынке, необходимо рассмотреть результат сравнения затрат на разработку данного программного продукта, а также полученный прирост чистой прибыли за год.

Сумма затрат на разработку меньше суммы годового экономического эффекта, поэтому можно сделать вывод, что такие инвестиции окупятся менее, чем за один год.

Таким образом, оценка экономической эффективности инвестиций производится при помощи расчёта рентабельности инвестиций (Return on Investment, ROI). Формула для расчёта ROI:

$$ROI = \frac{\Delta П_{ч}^{р} - З_{р}}{З_{р}} \cdot 100\% \quad (6.9)$$

где $\Delta П_{ч}^{р}$ – прирост чистой прибыли, полученной от реализации программного средства на рынке информационных технологий, р.;

$З_{р}$ – затраты на разработку и реализацию программного средства, р.

$$ROI = \frac{10\,720{,}00 - 6\,684{,}22}{6\,684{,}22} \cdot 100\% = 40{,}36\%$$

## 6.5 Вывод об экономической целесообразности реализации проектного решения

Проведённые расчёты технико-экономического обоснования позволяют сделать предварительный вывод о целесообразности разработки данного программного продукта. Общая сумма затрат на разработку и реализацию составила 6 684,22 Белорусских рублей, а отпускная цена была установлена на уровне 16,08 Белорусских рублей.

Прирост чистой прибыли за год, исходя из предполагаемого объёма продаж в размере 2500 расширенных версий в год, составляет 10 720,00

Белорусских рублей. Рентабельность инвестиций за год составляет 40,36%.

Это означает, что разработка данного программного продукта является целесообразной и реализация программного средства по установленной цене имеет смысл.

Однако, следует учитывать возможные риски, связанные с конкуренцией со стороны аналогов, что может привести к незамеченности продукта на рынке. Кроме того, высокая рентабельность связана с рисками, и расчётные результаты были получены при предполагаемом объёме продаж в 2500 копий в год.

Тем не менее, при поддержке проект может получить долгосрочное и успешное развитие, и количество проданных копий может превысить предполагаемое количество.

# ЗАКЛЮЧЕНИЕ

В рамках дипломного проекта была разработана платформа для анализа статистики игрока в компьютерной игре Dota 2. Этот процесс включал всесторонний анализ предметной области, изучение профильной литературы и существующих решений в сфере игровой аналитики. Для реализации приложения были выбраны оптимальные инструменты: язык C# с фреймворком ASP.NET Core для серверной части, документоориентированная база данных MongoDB, а также React для клиентской части. Данный выбор обеспечил производительность, стабильность и гибкость системы.

На этапе проектирования была создана подробная информационная и функциональная модель будущего приложения, что заложило прочную архитектурную основу. Разработанные архитектурные решения, включая трехслойную архитектуру для серверной части и принципы Feature-Sliced Design для клиентской, четко структурировали систему и определили взаимодействие её компонентов, в том числе с внешними API Steam, OpenDota, Stratz и сервисом AI (OpenRouter).

По результатам тестирования, программное средство полностью соответствует функциональным требованиям и готово к эксплуатации. Проведенные модульные тесты пользовательского интерфейса, а также системные проверки подтвердили надежность и корректность работы приложения.

Итогом дипломного проектирования стало создание высокоэффективной платформы для анализа статистики Dota 2, которая соответствует всем заявленным критериям. В перспективе планируется расширение функциональности приложения, в том числе внедрение новых аналитических метрик и интеграция с дополнительными игровыми сервисами.

# СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ


[1] Platform of Growth: An Analysis of Earnings and Social Media Engagement Among Esports Athletes [Электронный ресурс]. – Режим доступа: https://scholarcommons.sc.edu/cgi/viewcontent.cgi?article=8321&context=etd. – Дата доступа: 29.04.2025

[2] How Data Analytics is Shaping Competitive Esports [Электронный ресурс]. – Режим доступа: https://pley.gg/how-data-analytics-is-shaping-competitive-esports/. – Дата доступа: 29.04.2025

[3] History of Video Games [Электронный ресурс]. – Режим доступа: https://www.ebsco.com/research-starters/history/history-video-games. – Дата доступа: 29.04.2025

[4] The Gaming Industry in 2024 by the Numbers – A Review by GamesIndustry [Электронный ресурс]. – Режим доступа: https://app2top.com/news/the-gaming-industry-in-2024-by-the-numbers-a-review-by-gamesindustry-276003.html. – Дата доступа: 29.04.2025

[5] The Evolution of Dota 2 TI and Its Impact on Gameplay [Электронный ресурс]. – Режим доступа: https://gameflip.com/blog/the-evolution-of-dota-2-ti-and-its-impact-on-gameplay. – Дата доступа: 29.04.2025

[6] Web, Mobile or Desktop App – Which Is Right for Your Project? [Электронный ресурс]. – Режим доступа: https://denimdev.com.au/learn/web-mobile-or-desktop. – Дата доступа: 29.04.2025

[7] The Architect's Blueprint: 10 архитектурных стилей программного обеспечения и их паттерны [Электронный ресурс]. – Режим доступа: https://systems.education/software_styles_and_patterns_with_cheatsheet. – Дата доступа: 29.04.2025

[8] Сравнение микросервисной и монолитной архитектур [Электронный ресурс]. – Режим доступа: https://www.atlassian.com/ru/microservices/microservices-architecture/microservices-vs-monolith. – Дата доступа: 29.04.2025

[9] Клиент-серверная архитектура [Электронный ресурс]. – Режим доступа: https://servergate.ru/articles/klient-servernaya-arkhitektura/. – Дата доступа: 29.04.2025

[10] Что такое архитектура, управляемая событием (EDA)? [Электронный ресурс]. – Режим доступа: https://aws.amazon.com/ru/what-is/eda/. – Дата доступа: 29.04.2025

[11] JavaScript. [Электронный ресурс]. – Режим доступа: https://blog.skillfactory.ru/glossary/javascript/. – Дата доступа: 29.04.2025

[12] C# Programming: What It Is, How It's Used + How to Learn It. [Электронный ресурс]. – Режим доступа: https://www.coursera.org/articles/c-sharp. – Дата доступа: 29.04.2025

[13] MongoDB. [Электронный ресурс]. – Режим доступа https://blog.skillfactory.ru/glossary/mongodb/. – Дата доступа: 29.04.2025

[14] Как работает Docker: подробный гайд от техлида. [Электронный ресурс]. – Режим доступа https://skillbox.ru/media/code/kak-rabotaet-docker-podrobnyy-gayd-ot-tekhlida/. – Дата доступа: 29.04.2025

[15] MongoDB: что это за СУБД, плюсы, минусы, подводные камни. [Электронный ресурс]. – Режим доступа https://skillbox.ru/media/code/mongodb-chto-eto-za-subd-plyusy-minusy-podvodnye-kamni/. – Дата доступа: 29.04.2025

[16] What is three-tier architecture? [Электронный ресурс]. – Режим доступа https://www.ibm.com/think/topics/three-tier-architecture. – Дата доступа: 29.04.2025

[17] Обзор (FSD) [Электронный ресурс]. – Режим доступа https://feature-sliced.github.io/documentation/ru/docs/get-started/overview. – Дата доступа: 29.04.2025

# ПРИЛОЖЕНИЕ А
# (обязательное)
# Результат проверки в системе «Антиплагиат»

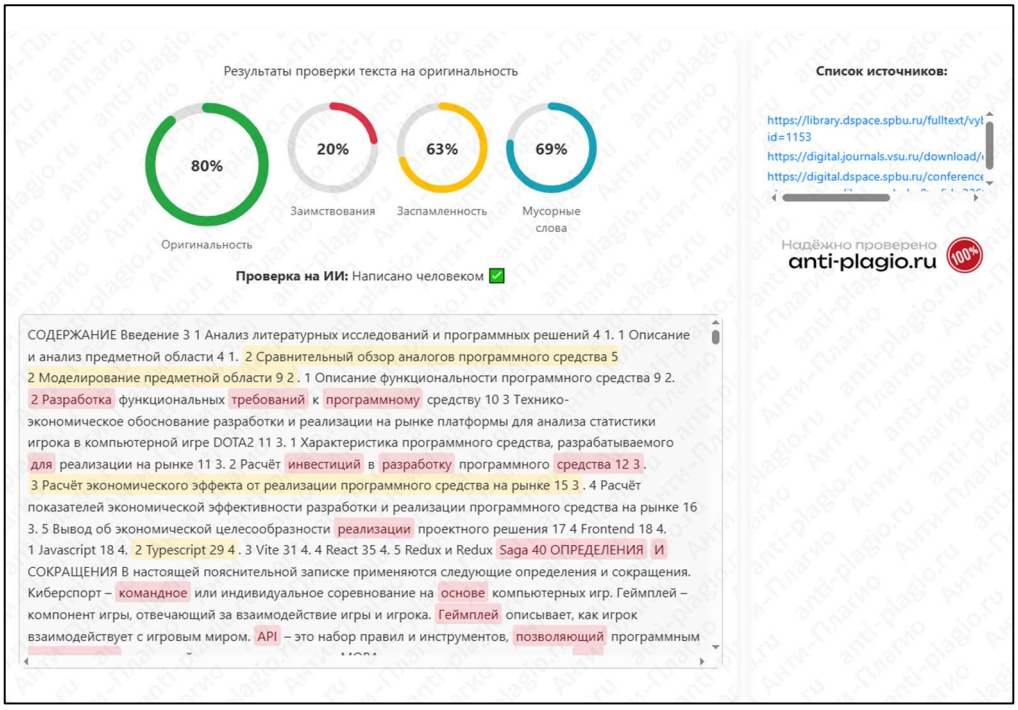



Рисунок А.1 – Результат проверки в системе «Антиплагиат»

# ПРИЛОЖЕНИЕ Б
## (обязательное)
## Исходный код программы

```
using System.Net.Http.Headers;
using System.Text;
using Microsoft.Extensions.Configuration;
using Newtonsoft.Json;

namespace DotaMaster.Data.Repositories
{
    public class AiRepository
    {
        private readonly IConfiguration _configuration;
        private readonly HttpClient _httpClient;
        private readonly string _apiKey;
        private const string _url =
"https://openrouter.ai/api/v1/chat/completions";
        private const string _model = "deepseek/deepseek-
chat-v3-0324:free";

        public AiRepository(IConfiguration configuration,
HttpClient httpClient)
        {
            _configuration = configuration;
            _httpClient = httpClient;
            _apiKey = _configuration["AiApiKey"]
                ?? throw new ArgumentNullException("Ai api
key is null!");
        }

        public async Task<string> AskAi(string question)
        {
            var response = await SendRequest(question);
            response.EnsureSuccessStatusCode();
            var jsonResponse = await
response.Content.ReadAsStringAsync();
            dynamic responseObj =
JsonConvert.DeserializeObject(jsonResponse)
                ?? throw new ArgumentNullException("Can not
parse AskAi response");

            return
responseObj?.choices?[0]?.message?.content?.ToString()
                ?? throw new ArgumentNullException("AskAi
response content is null");
        }

        private async Task<HttpResponseMessage>
SendRequest(string question)
```

```
            {
                var content = new
                {
                    model = _model,
                    messages = new[]
                    {
                        new
                        {
                            role = "user",
                            content = question
                        }
                    },
                    provider = new
                    {
                        order = new[]
                        {
                            "Chutes",
                        }
                    }
                };

                string             jsonRequestBody             =
JsonConvert.SerializeObject(content);
                var          httpRequest          =          new
HttpRequestMessage(HttpMethod.Post, _url)
                {
                    Content = new StringContent(jsonRequestBody,
Encoding.UTF8, "application/json")
                };
                httpRequest.Headers.Authorization     =     new
AuthenticationHeaderValue("Bearer", _apiKey);
                return await _httpClient.SendAsync(httpRequest);
            }
        }
    }
            public async Task<PickInfo> GetPickAsync(string rank,
string rankBracket, long dotaId, int heroId, string position,
List<int> alliedHeroes, List<int> enemyHeroes, int take = 125)
            {
                string       graphqlQuery       =       @"query
GetHeroWinrates($heroId: Short, $heroIds: [Short], $take: Int,
$rank: RankBracket, $rankBracket: RankBracketBasicEnum, $pos:
MatchPlayerPositionType) {
                  heroStats {
                    winWeek(
                      heroIds: [$heroId]
                      gameModeIds: [ALL_PICK_RANKED]
                      bracketIds: [$rank]
                      positionIds: [$pos]
                    ) {
                      matchCount
```

```
                winCount
              }
              matchUp(
                heroId: $heroId
                heroIds: $heroIds
                bracketBasicIds: [$rankBracket]
                take: $take
              ) {
                heroId
                vs {
                  heroId1
                  heroId2
                  winsAverage
                }
                with {
                  heroId1
                  heroId2
                  winsAverage
                }
              }
            }
          }";

          var requestBody = new
          {
              query = graphqlQuery,
              variables = new { heroId, rank, rankBracket,
take, pos = position }
          };

          var          response          =          await
SendGraphqlRequest(requestBody);
          response.EnsureSuccessStatusCode();
          var         jsonResponse         =         await
response.Content.ReadAsStringAsync();
          var                  parsed                  =
JsonConvert.DeserializeObject<Dictionary<string,
object>>(jsonResponse)
              ?? throw new ArgumentNullException("Can not
parse GetPickAnalyzeAsync response");
          var              winratesResponse              =
(((JObject)parsed["data"])
              ["heroStats"]       ??       throw       new
BadRequestException("GetPickAsync: Invalid input"))
              .ToObject<WinratesResponse>()  ??  throw  new
ArgumentNullException("Can not parse to WinratesResponse");
          ValidateHeroCounts(alliedHeroes, enemyHeroes);

          var alliedWinrates = alliedHeroes.Select(ally =>
new HeroWrInfo
          {
              HeroId = ally,
```

```
                    Winrate                                          =
winratesResponse.MatchUp.First().With.Single(h  =>  h.HeroId2  ==
ally).WinsAverage
                }).ToList();

                var enemyWinrates = enemyHeroes.Select(enemy =>
new HeroWrInfo
                {
                    HeroId = enemy,
                    Winrate                                          =
winratesResponse.MatchUp.First().Vs.Single(h   =>   h.HeroId2   ==
enemy).WinsAverage
                }).ToList();

                return new PickInfo
                {
                    HeroId = heroId,
                    HeroWinrate                                      =
(double)winratesResponse.WinWeek.Last().WinCount /

winratesResponse.WinWeek.Last().MatchCount,
                    HeroWrWithAlliedHeroes = alliedWinrates,
                    HeroWrWithEnemyHeroes = enemyWinrates
                };
            }

    namespace DotaMaster.Data.Repositories
    {
        public class ProfileRepository
        {
            private readonly string _steamApiKey;
            private readonly string _stratzApiKey;
            private readonly IConfiguration _configuration;
            private readonly HttpClient _httpClient;
            private readonly HeroRepository _heroRepository;
            private     const     string     _openDotaUrl     =
"https://api.opendota.com/api";
            private    const    string    _stratzGraphqlUrl    =
"https://api.stratz.com/graphql";

            public              ProfileRepository(IConfiguration
configuration,     HttpClient     httpClient,     HeroRepository
heroRepository)
            {
                _heroRepository = heroRepository;
                _httpClient = httpClient;
                _configuration = configuration;
                _steamApiKey = _configuration["Steam:ApiKey"]
                    ?? throw new ArgumentNullException("Steam api
key is null!");
                _stratzApiKey = _configuration["StratzApiKey"]
```

```
                    ?? throw new ArgumentNullException("Stratz
api key is null!");
            }

            public async Task<SteamProfile>
GetSteamUserProfileAsync(string steamId)
            {
                string url =
$"https://api.steampowered.com/ISteamUser/GetPlayerSummaries/v2/
?key={_steamApiKey}&steamids={steamId}";
                var dotaId =
SteamIdConverter.SteamIdToDotaId(SteamIdConverter.GetIDFromCommu
nity(steamId));
                var response = await _httpClient.GetAsync(url);
                response.EnsureSuccessStatusCode();
                var jsonResponse = await
response.Content.ReadAsStringAsync();
                var parsed =
JsonConvert.DeserializeObject<Dictionary<string,
object>>(jsonResponse)
                    ?? throw new ArgumentNullException("Can not
parse GetSteamUserProfileAsync response");
                var player = ((((JObject)parsed["response"]
                    ?? throw new ArgumentNullException("Can not
parse GetSteamUserProfileAsync response: 'response' is null"))
                    ["players"]
                    ?? throw new ArgumentNullException("Can not
parse GetSteamUserProfileAsync response: 'players' is null"))
                    .ToObject<Player[]>()
                    ?? throw new ArgumentNullException("Can not
parse GetSteamUserProfileAsync response: can not parse to
'Player'"))[0];
                return new SteamProfile()
                {
                    AvatarUrl = player.AvatarUrl,
                    DotaId = dotaId,
                    SteamId = player.SteamId,
                    Username = player.Username
                };
            }

            public async Task<BasicInfo>
GetBasicInfoAsync(string steamId)
            {
                var dotaId =
SteamIdConverter.SteamIdToDotaId(SteamIdConverter.GetIDFromCommu
nity(steamId));
                const string query = @"
                    query BasicInfoQuery($dotaId: Long!) {
                      player(steamAccountId: $dotaId) {
                        steamAccount {
                          seasonRank
```

```
                        }
                        matchCount
                        firstMatchDate
                        winCount
                        matchCount
                        isFollowed
                    }
                }";

            var requestBody = new
            {
                query,
                variables = new { dotaId }
            };
            var response = await
SendGraphqlRequest(requestBody);
            response.EnsureSuccessStatusCode();

            var jsonResponse = await
response.Content.ReadAsStringAsync();
            var parsed =
JsonConvert.DeserializeObject<Dictionary<string,
object>>(jsonResponse)
                ?? throw new ArgumentNullException("Can not
parse GetBasicInfoAsync response");
            var basicInfoResponse =
((((JObject)parsed["data"])
                ?? throw new ArgumentNullException("Can not
parse GetBasicInfoAsync response: 'data' is null"))
                ["player"]
                ?? throw new ArgumentNullException("Can not
parse GetBasicInfoAsync response: 'player' is null"))
                .ToObject<BasicInfoResponse>()
                ?? throw new ArgumentNullException("Can not
parse GetBasicInfoAsync response: can not parse to
'BasicInfoResponse'");

            return new BasicInfo()
            {
                FirstMatchDate =
DateOnly.FromDateTime(DateTimeOffset.FromUnixTimeSeconds(basicIn
foResponse.FirstMatchDate).UtcDateTime),
                IsDotaPlusSub =
basicInfoResponse.IsDotaPlusSub,
                Loses = basicInfoResponse.MatchCount -
basicInfoResponse.Wins,
                Rank =
basicInfoResponse.SteamAccount.SeasonRank.ToString(),
                Wins = basicInfoResponse.Wins
            };
        }
```

```
            public async Task<Records> GetRecordsAsync(string
steamId)
            {
                var                    dotaId                    =
SteamIdConverter.SteamIdToDotaId(SteamIdConverter.GetIDFromCommu
nity(steamId));
                string                    url                    =
$"{_openDotaUrl}/players/{dotaId}/totals";
                var response = await _httpClient.GetAsync(url);
                response.EnsureSuccessStatusCode();
                var               recordsResponse               =
JsonConvert.DeserializeObject<RecordsResponse[]>(await
response.Content.ReadAsStringAsync())
                    ?? throw new ArgumentNullException("Can not
parse GetRecordsAsync response");

                var records = new Records();

                foreach (var record in recordsResponse)
                {
                    if (record.Field == "kills")
                    {
                        records.Kills = record.TotalCount;
                        records.AvgKills                        =
(int)Math.Ceiling(record.TotalCount / (double)record.Matches);
                    }
                    else if (record.Field == "deaths")
                    {
                        records.Deaths = record.TotalCount;
                        records.AvgDeaths                       =
(int)Math.Ceiling(record.TotalCount / (double)record.Matches);
                    }
                    else if (record.Field == "assists")
                    {
                        records.Assists = record.TotalCount;
                        records.AvgAssists                      =
(int)Math.Ceiling(record.TotalCount / (double)record.Matches);
                    }
                    else if (record.Field == "gold_per_min")
                    {
                        records.Gold = record.TotalCount;
                        records.AvgGold                         =
(int)Math.Ceiling(record.TotalCount / (double)record.Matches);
                    }
                    else if (record.Field == "xp_per_min")
                    {
                        records.Xp = record.TotalCount;
                        records.AvgXp                           =
(int)Math.Ceiling(record.TotalCount / (double)record.Matches);
                    }
                    else if (record.Field == "last_hits")
                    {
```

```
                        records.LastHits = record.TotalCount;
                        records.AvgLastHits                    =
(int)Math.Ceiling(record.TotalCount / (double)record.Matches);
                    }
                    else if (record.Field == "denies")
                    {
                        records.Denies = record.TotalCount;
                        records.AvgDenies                      =
(int)Math.Ceiling(record.TotalCount / (double)record.Matches);
                    }
                    else if (record.Field == "hero_damage")
                    {
                        records.HeroDamage = record.TotalCount;
                        records.AvgHeroDamage                  =
(int)Math.Ceiling(record.TotalCount / (double)record.Matches);
                    }
                    else if (record.Field == "tower_damage")
                    {
                        records.TowerDamage = record.TotalCount;
                        records.AvgTowerDamage                 =
(int)Math.Ceiling(record.TotalCount / (double)record.Matches);
                    }
                    else if (record.Field == "hero_healing")
                    {
                        records.HeroHealing = record.TotalCount;
                        records.AvgHeroHealing                 =
(int)Math.Ceiling(record.TotalCount / (double)record.Matches);
                    }
                }
                return records;
            }

            public       async       Task<IEnumerable<HeroStat>>
GetRecentHeroesStatsAsync(string steamId)
            {
                var                   dotaId                   =
SteamIdConverter.SteamIdToDotaId(SteamIdConverter.GetIDFromCommu
nity(steamId));
                string            recentMatchesUrl             =
$"{_openDotaUrl}/players/{dotaId}/recentMatches";
                var      recentMatchesResponse      =      await
_httpClient.GetAsync(recentMatchesUrl);
                var               recentMatches                =
JsonConvert.DeserializeObject<RecentMatchIdResponse[]>(await
recentMatchesResponse.Content.ReadAsStringAsync())
                    ?? throw new ArgumentNullException($"Profile
{steamId} is private!");

                string query = @"
                    query  GetUser($userId:  Long!,  $matchIds:
[Long]) {
                      player(steamAccountId: $userId) {
```

```
                        heroesPerformance(request:    {matchIds:
$matchIds}) {
                          heroId
                          hero {
                            name
                          }
                          matchCount
                          avgKills
                          avgDeaths
                          duration
                          avgAssists
                          goldPerMinute
                          experiencePerMinute
                          winCount
                          imp
                          best
                          positionScore {
                            score
                            matchCount
                            imp
                          }
                        }
                      }
                    }";

                var  matchIds  =  recentMatches.Select(match  =>
match.MatchId).ToArray();
                var requestBody = new
                {
                    query,
                    variables = new { userId = dotaId, matchIds
}
                };
                var          response          =          await
SendGraphqlRequest(requestBody);
                response.EnsureSuccessStatusCode();
                var        responseContent        =        await
response.Content.ReadAsStringAsync();

                var             heroStatsResponse             =
JsonConvert.DeserializeObject<HeroStatsResponse>(responseContent
)
                    ?? throw new ArgumentNullException("Can not
parse GetRecentHeroesStatsAsync response: response is null");

                return
heroStatsResponse.Data.Player.HeroStats.Select(heroStat  =>  new
HeroStat
                {
                    HeroId = heroStat.HeroId,
                    Name = heroStat.Hero.Name,
                    MatchCount = heroStat.MatchCount,
```

```
                    WinCount = heroStat.WinCount,
                    AvgAssists = heroStat.AvgAssists,
                    AvgDeaths = heroStat.AvgDeaths,
                    AvgKills = heroStat.AvgKills,
                    AvgGpm = heroStat.Gpm,
                    AvgXpm = heroStat.Xpm,
                    Impact = heroStat.Impact
                }).ToList();
            }

            public async Task<IEnumerable<MatchBasicInfo>>
GetMatchesInfoAsync(string steamId, int? heroId, int limit = 15,
int offset = 0)
            {
                var dotaId =
SteamIdConverter.SteamIdToDotaId(SteamIdConverter.GetIDFromCommu
nity(steamId));
                string url =
$"{_openDotaUrl}/players/{dotaId}/matches?game_mode=22&limit={li
mit}&offset={offset}";
                if (heroId != null)
                {
                    url += $"&hero_id={heroId}";
                }

                var response = await _httpClient.GetAsync(url);
                response.EnsureSuccessStatusCode();

                var matchesJson = await
response.Content.ReadAsStringAsync();
                var matches =
JsonConvert.DeserializeObject<MatchBasicInfoResponse[]>(matchesJ
son)
                    ?? throw new ArgumentNullException("Can not
parse GetMatchesInfoAsync response: response is null");

                var heroes = await
_heroRepository.GetHeroesOpendota();

                return matches.Select(match => new MatchBasicInfo
                {
                    MatchId = match.MatchId,
                    Hero = heroes.First(h => h.Id ==
match.HeroId),
                    IsWin = match.PlayerSlot < 100?
match.RadiantWin : !match.RadiantWin,
                    Duration =
TimeSpan.FromSeconds(match.Duration),
                    Kills = match.Kills,
                    Skill = match.Skill,
                    StartTime =
DateTimeOffset.FromUnixTimeSeconds(match.StartTime).UtcDateTime,
```

```
                    Deaths = match.Deaths,
                    Assists = match.Assists,
                    AverageRank = match.AverageRank ?? 0
                }).ToList();
            }

            private async Task<HttpResponseMessage>
SendGraphqlRequest(object requestBody)
            {
                string jsonRequestBody =
JsonConvert.SerializeObject(requestBody);
                var httpRequest = new
HttpRequestMessage(HttpMethod.Post, _stratzGraphqlUrl)
                {
                    Content = new StringContent(jsonRequestBody,
Encoding.UTF8, "application/json")
                };
                httpRequest.Headers.Authorization = new
AuthenticationHeaderValue("Bearer", _stratzApiKey);

httpRequest.Headers.UserAgent.ParseAdd("STRATZ_API");
                return await _httpClient.SendAsync(httpRequest);
            }
        }
    }
```